

Demystify the neural network

- The (artificial) neural network and deep learning in general receive a lot of attention and hype.
- Neural networks are powerful but are **not** magical.

Demystify the neural network

- They are statistical models, based on the principals and tools you have seen in this course.
- In fact, the field of neural networks is considered to have started in the 1940s!
 - Mathematical model of a neuron McCulloch and Pitts 1943
 - Rosenblatt perceptron learning algorithm 1957

Demystify the neural network

- They are statistical models, based on the principals and tools you have seen in this course.
- You have all tools necessary to fit neural networks from scratch!

Demystify the neural network

- They are statistical models, based on the principals and tools you have seen in this course.
- You have all tools necessary to fit neural networks from scratch!

In this presentation, we will focus on how the topics from linear and generalized linear models are the **foundation** of the neural network.

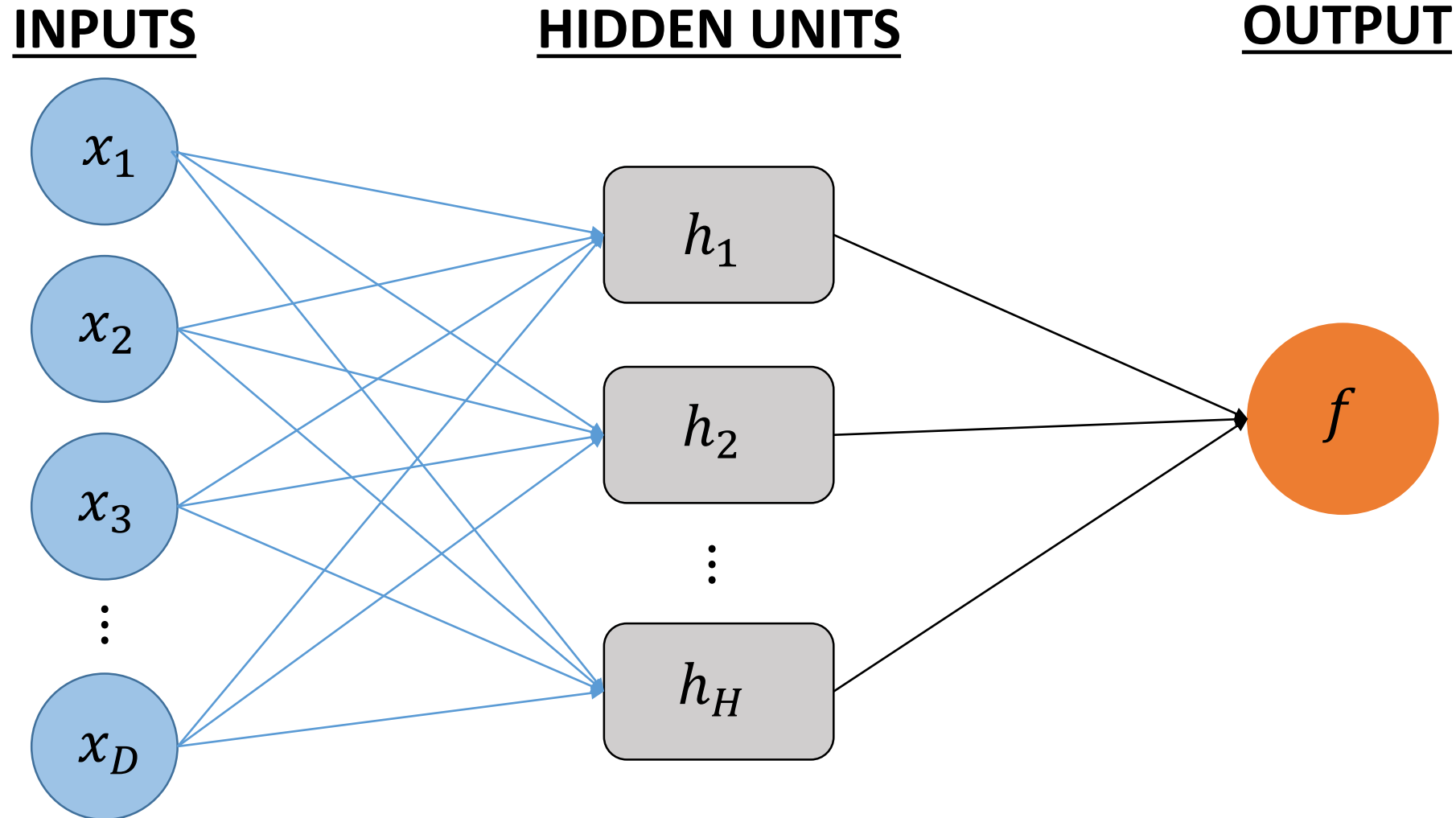
Neural network architecture and nomenclature

- A neural network is a series of functional transformations.
- The output is modeled through an intermediary set of **unobserved** variables called **hidden units**.
- The **hidden units** are usually called **neurons**.

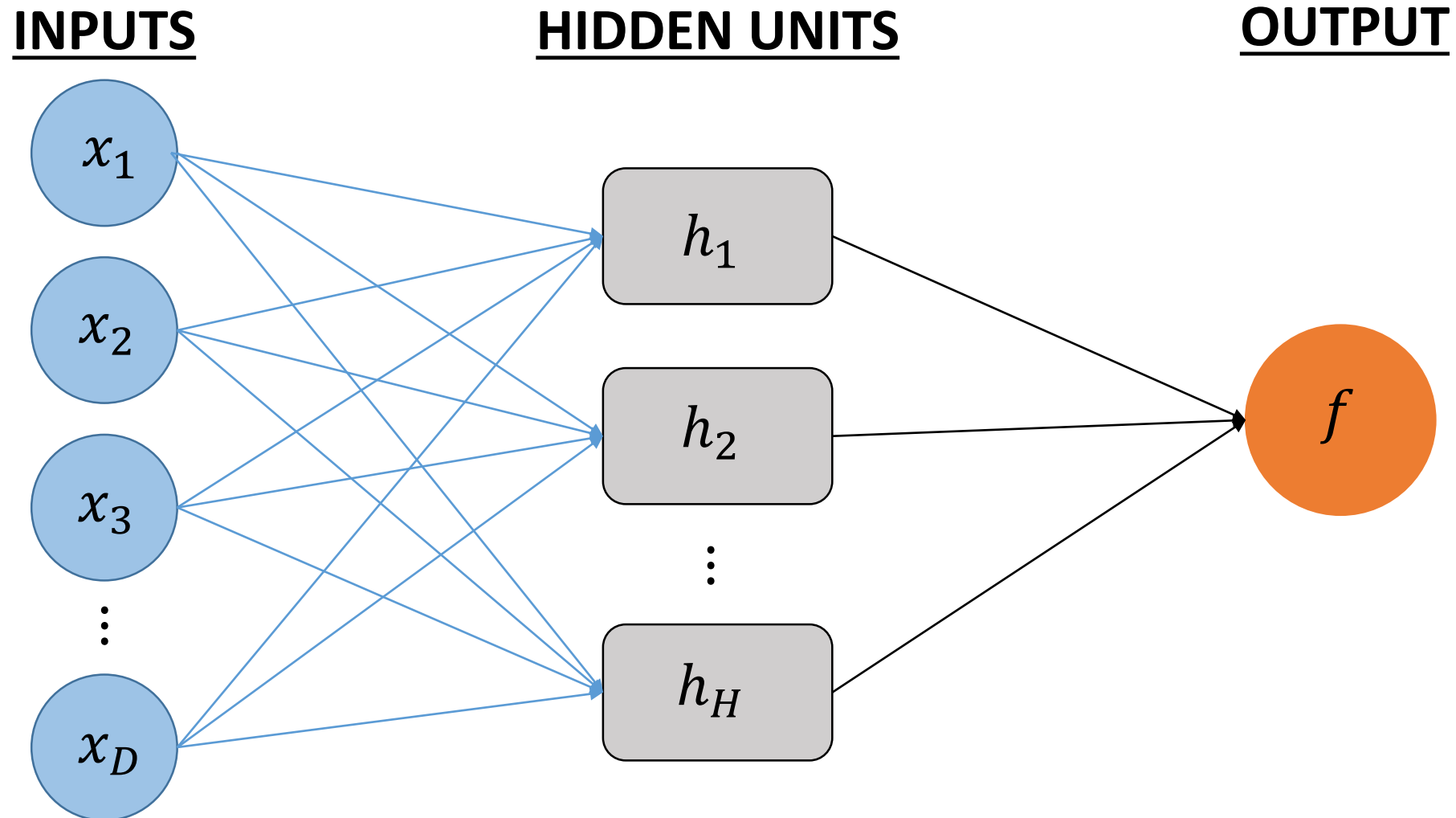
Neural network architecture and nomenclature

- The hidden units are derived as linear combinations of the inputs, transformed through a non-linear function.
- The non-linear transformation function is usually called an activation function.
- The response is modeled as a linear combination of the hidden units.

Schematic of a single hidden layer neural network for a continuous response

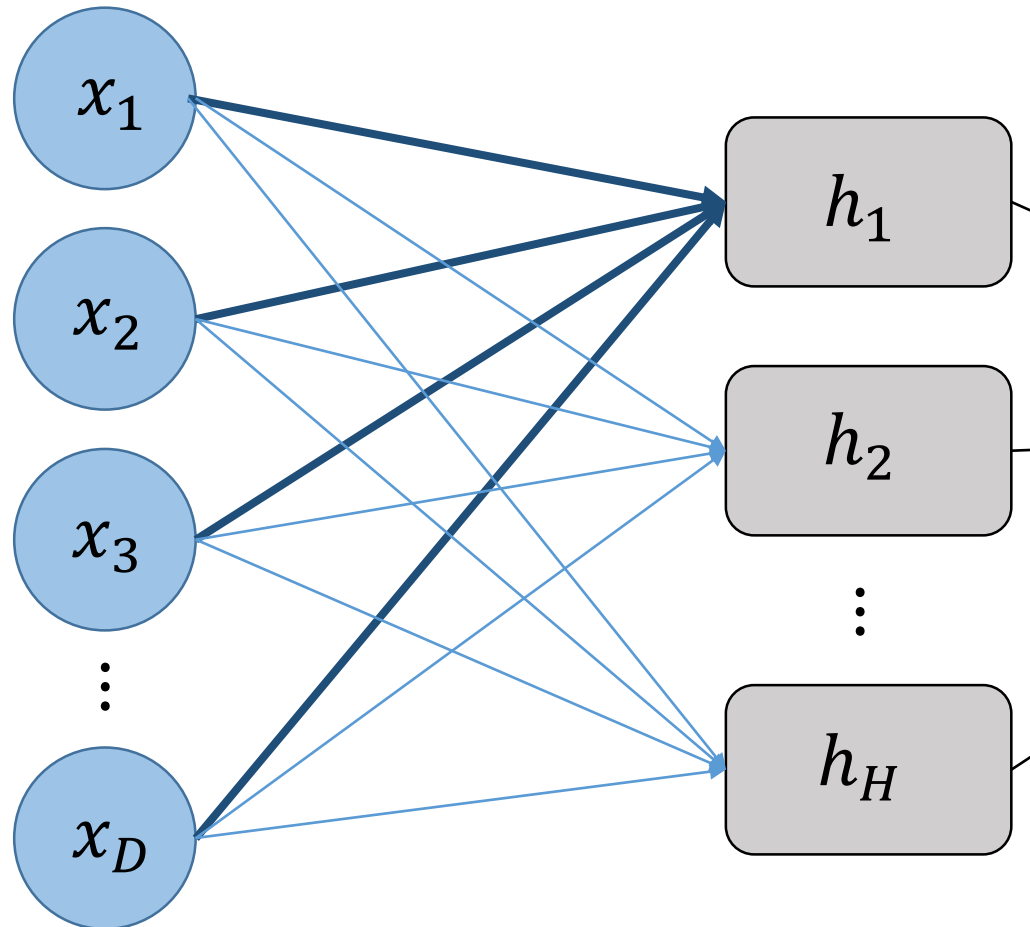


Hidden units are **non-linear** functions of **linear** combinations of the inputs



Hidden unit 1

INPUTS



Linear combination of the inputs:

$$\beta_{0,1} + \sum_{d=1}^D x_{n,d} \beta_{d,1}$$

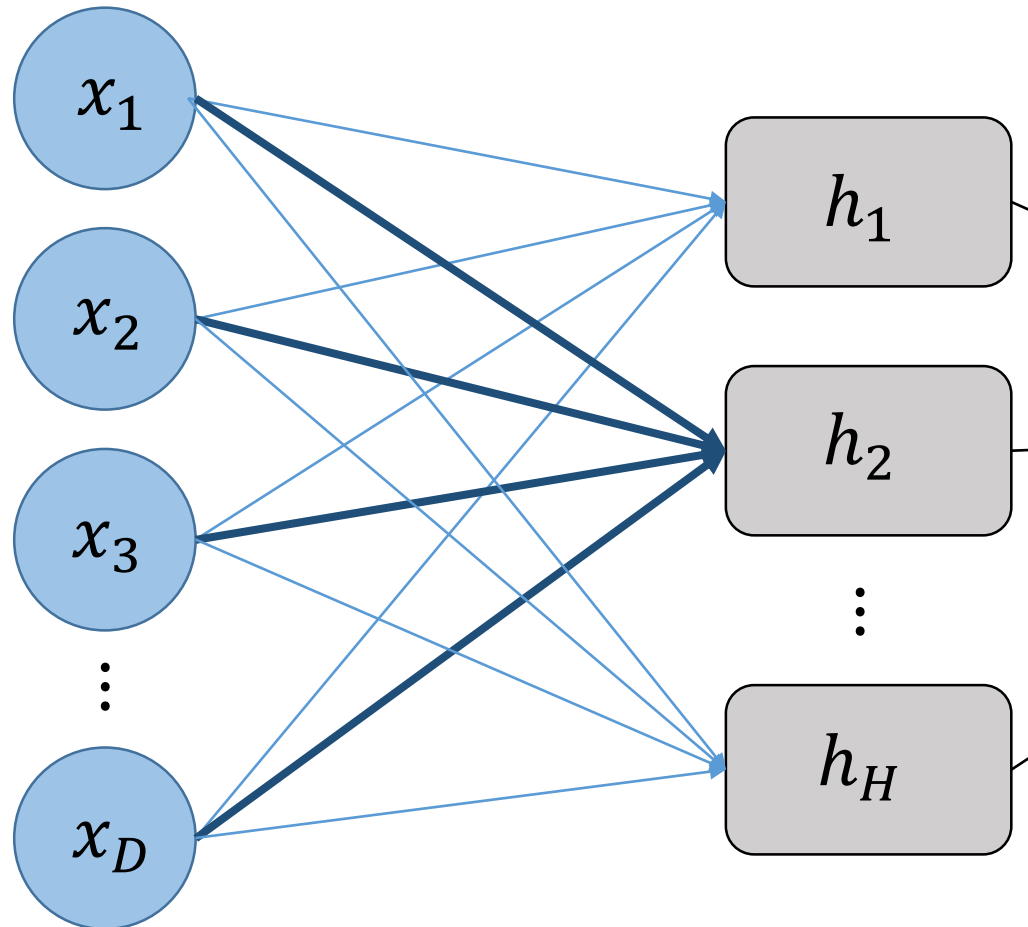
Pass through a **non-linear** function:

$$h_{n,1}(\mathbf{x}_{n,:}) = g \left(\beta_{0,1} + \sum_{d=1}^D x_{n,d} \beta_{d,1} \right)$$

Likewise for hidden unit 2

INPUTS

HIDDEN UNITS



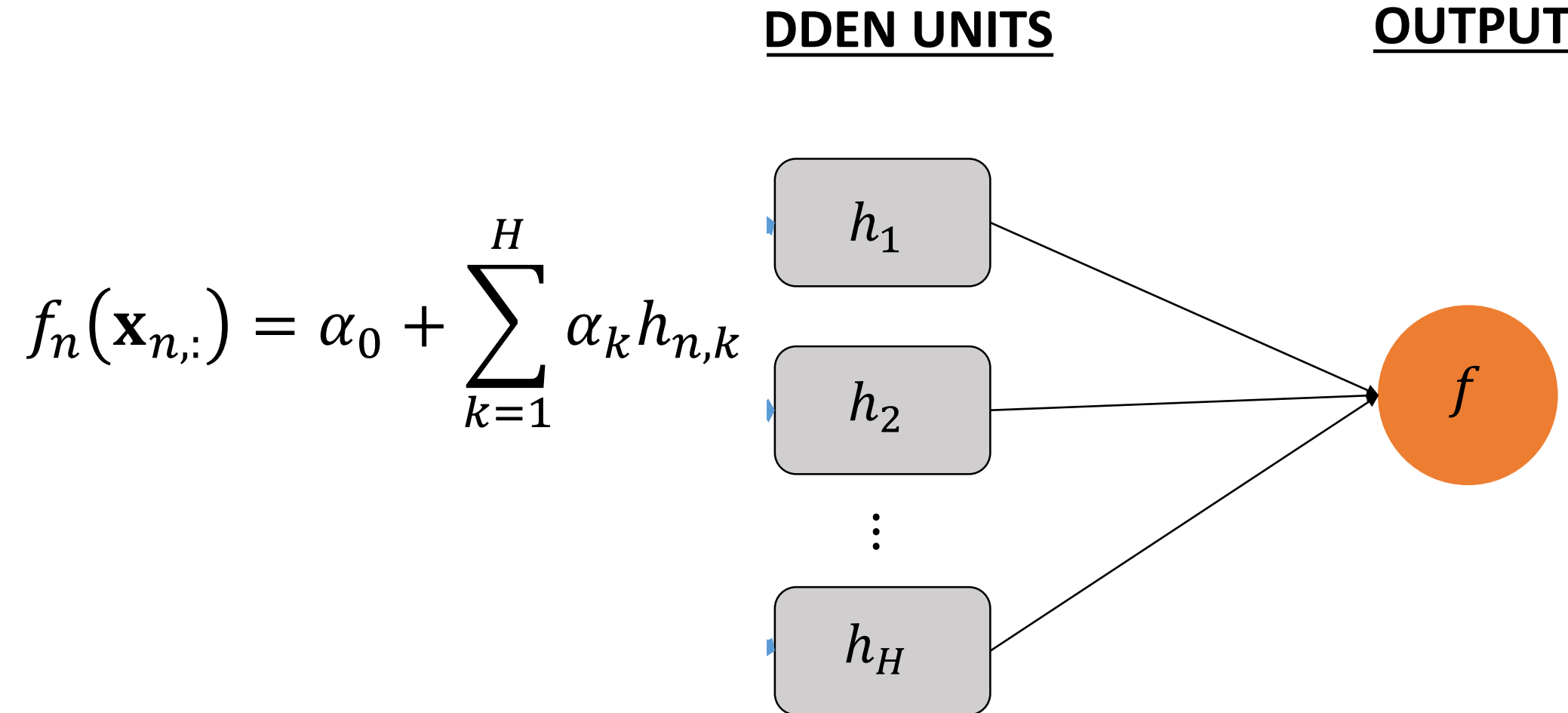
Linear combination of the inputs:

$$\beta_{0,2} + \sum_{d=1}^D x_{n,d} \beta_{d,2}$$

Pass through a **non-linear** function:

$$h_{n,2}(\mathbf{x}_{n,:}) = g \left(\beta_{0,2} + \sum_{d=1}^D x_{n,d} \beta_{d,2} \right)$$

For a continuous response, the output is a **linear** combination of the hidden units



Hidden unit parameter terminology for the k -th hidden unit

$$h_{n,k}(\mathbf{x}_{n,:}) = g \left(\underset{\substack{\uparrow \\ \text{BIAS}}}{\beta_{0,k}} + \sum_{d=1}^D x_{n,d} \underset{\substack{\uparrow \\ d\text{-th WEIGHT}}}{\beta_{d,k}} \right)$$

Rewrite **linear** combination of the inputs by defining the “fake” input $x_{n,d=0} = 1$

$$h_{n,k}(\mathbf{x}_{n,:}) = g \left(\sum_{d=0}^D x_{n,d} \beta_{d,k} \right)$$

Rewrite **linear** combination of the inputs by defining the “fake” input $x_{n,d=0} = 1$

$$h_{n,k}(\mathbf{x}_{n,:}) = g \left(\sum_{d=0}^D x_{n,d} \beta_{d,k} \right)$$

Define a column vector $\boldsymbol{\beta}_k$ of the k -th hidden unit’s parameters:

$$\boldsymbol{\beta}_k = \begin{bmatrix} \beta_{d=0,k} \\ \beta_{d=1,k} \\ \vdots \\ \beta_{d=D,k} \end{bmatrix}$$

Rewrite **linear** combination of the inputs by defining the “fake” input $x_{n,d=0} = 1$

$$h_{n,k}(\mathbf{x}_{n,:}) = g\left(\sum_{d=0}^D x_{n,d} \beta_{d,k}\right)$$

Rewrite the summation as an inner-product between a row vector $\mathbf{x}_{n,:}$ and a column vector $\boldsymbol{\beta}_k$:

$$h_{n,k}(\mathbf{x}_{n,:}) = g(\mathbf{x}_{n,:} \boldsymbol{\beta}_k)$$

Rewrite **linear** combination of the inputs by defining the “fake” input $x_{n,d=0} = 1$

$$h_{n,k}(\mathbf{x}_{n,:}) = g\left(\sum_{d=0}^D x_{n,d} \beta_{d,k}\right)$$

Rewrite the summation as an inner-product between a row vector $\mathbf{x}_{n,:}$ and a column vector $\boldsymbol{\beta}_k$:

$$h_{n,k}(\mathbf{x}_{n,:}) = g(\mathbf{x}_{n,:} \boldsymbol{\beta}_k)$$

We have worked with inner-products before!

The linear combinations of the inputs are essentially linear models!

- If we have N training points of D inputs, we can assemble those inputs into the $N \times (D + 1)$ design matrix, $\mathbf{X}$ (include the intercept column of 1s).
- The k -th hidden unit's parameters are stored in a $(D + 1) \times 1$ column vector $\boldsymbol{\beta}_k$.
- The linear combination of the inputs for the k -th hidden unit:

$$\boldsymbol{\eta}_k = \mathbf{X}\boldsymbol{\beta}_k$$

There are a wide variety of non-linear transformation functions to use

- For now, we will use the general notation of $g(\cdot)$.
- The k -th hidden unit's response is:

$$\mathbf{h}_k = g(\boldsymbol{\eta}_k) = g(\mathbf{X}\boldsymbol{\beta}_k)$$

The output layer

- Assemble the hidden unit variables into a matrix:

$$\mathbf{H} = [\mathbf{h}_1 \quad \mathbf{h}_2 \quad \cdots \quad \mathbf{h}_H]$$

- The output layer parameters, $\{\alpha_0, \alpha_1, \alpha_2, \dots, \alpha_H\}$, are structured into a $H \times 1$ column vector $\boldsymbol{\alpha}$ and the scalar intercept, α_0 .
- The response is a linear model relative to the hidden units!

$$\mathbf{f} = \alpha_0 + \mathbf{H}\boldsymbol{\alpha}$$

The output layer – check dimensionality

- Assemble the hidden unit variables into a matrix:

$$\mathbf{H} = [\mathbf{h}_1 \quad \mathbf{h}_2 \quad \cdots \quad \mathbf{h}_H] \quad N \times H \text{ matrix}$$

- The output layer parameters, $\{\alpha_0, \alpha_1, \alpha_2, \dots, \alpha_H\}$, are structured into a $H \times 1$ column vector $\boldsymbol{\alpha}$ and the scalar intercept, α_0 .

$\boldsymbol{\alpha}$ is an $H \times 1$ column vector

- The response is a linear model relative to the hidden units!

$$\mathbf{f} = \alpha_0 + \boxed{\mathbf{H}\boldsymbol{\alpha}} \quad \begin{array}{c} (N \times H) \times (H \times 1) \\ \downarrow \\ N \times 1 \end{array}$$

All hidden units can be calculated at once with matrix math!

- Bind the H hidden unit parameter vectors into a matrix:

$$\mathbf{B} = [\boldsymbol{\beta}_1 \quad \boldsymbol{\beta}_2 \quad \cdots \quad \boldsymbol{\beta}_H]$$

- The transformed hidden units can then be calculated as:

$$\mathbf{H} = g(\mathbf{XB})$$

All hidden units can be calculated at once with matrix math!

- Bind the H hidden unit parameter vectors into a matrix:

$$\mathbf{B} = [\boldsymbol{\beta}_1 \quad \boldsymbol{\beta}_2 \quad \cdots \quad \boldsymbol{\beta}_H] \quad (D + 1) \times H \text{ matrix}$$

- The transformed hidden units can then be calculated as:

$$\mathbf{H} = g(\mathbf{XB}) \quad \begin{matrix} (N \times (D + 1)) \times ((D + 1) \times H) \\ \downarrow \\ N \times H \end{matrix}$$

A neural network is just a series of matrix operations!

- Neural network response calculated with output layer intercept (bias) α_0 and column vector of “weights” α :

$$\mathbf{f} = \alpha_0 + \mathbf{H}\alpha = \alpha_0 + [g(\mathbf{XB})]\alpha$$

Let's define a simple single hidden layer neural network with just 2 hidden units

```
### specify the hidden unit parameters
```

```
b1 <- c(0.2, 1.2)
```

```
b2 <- c(-0.1, -0.8)
```

```
### set the input to be evenly spaced between -3 and +3
```

```
x <- seq(-3, 3, length.out = 25)
```

```
### create the design matrix
```

```
Xmat <- model.matrix(~ x, data = data.frame(x = x))
```

```
### calculate the hidden unit linear predictors
```

```
Bmat <- cbind(b1, b2)
```


Let's define a simple single hidden layer neural network with just 2 hidden units

```
### specify the hidden unit parameters
b1 <- c(0.2, 1.2)
b2 <- c(-0.1, -0.8)

### set the input to be evenly spaced between -3 and +3
x <- seq(-3, 3, length.out = 25)

### create the design matrix
Xmat <- model.matrix(~ x, data = data.frame(x = x))

### calculate the hidden unit linear predictors
Bmat <- cbind(b1, b2)
```

What's the dimensionality of Bmat?

Let's define a simple single hidden layer neural network with just 2 hidden units

```
### specify the hidden unit parameters
```

```
b1 <- c(0.2, 1.2)
```

```
b2 <- c(-0.1, -0.8)
```

```
### set the input to be evenly spaced between -3 and +3
```

```
x <- seq(-3, 3, length.out = 25)
```

```
### create the design matrix
```

```
Xmat <- model.matrix(~ x, data = data.frame(x = x))
```

```
### calculate the hidden unit linear predictors
```

```
Bmat <- cbind(b1, b2)
```

```
> Bmat
```

	b1	b2
[1,]	0.2	-0.1
[2,]	1.2	-0.8

Calculate the linear combinations of the inputs per hidden unit

```
> Amat <- Xmat %*% Bmat
```

```
> dim(Amat)
```

```
[1] 25  2
```

```
> Amat %>% head()
```

```
      b1  b2
```

```
1 -3.4 2.3
```

```
2 -3.1 2.1
```

```
3 -2.8 1.9
```

```
4 -2.5 1.7
```

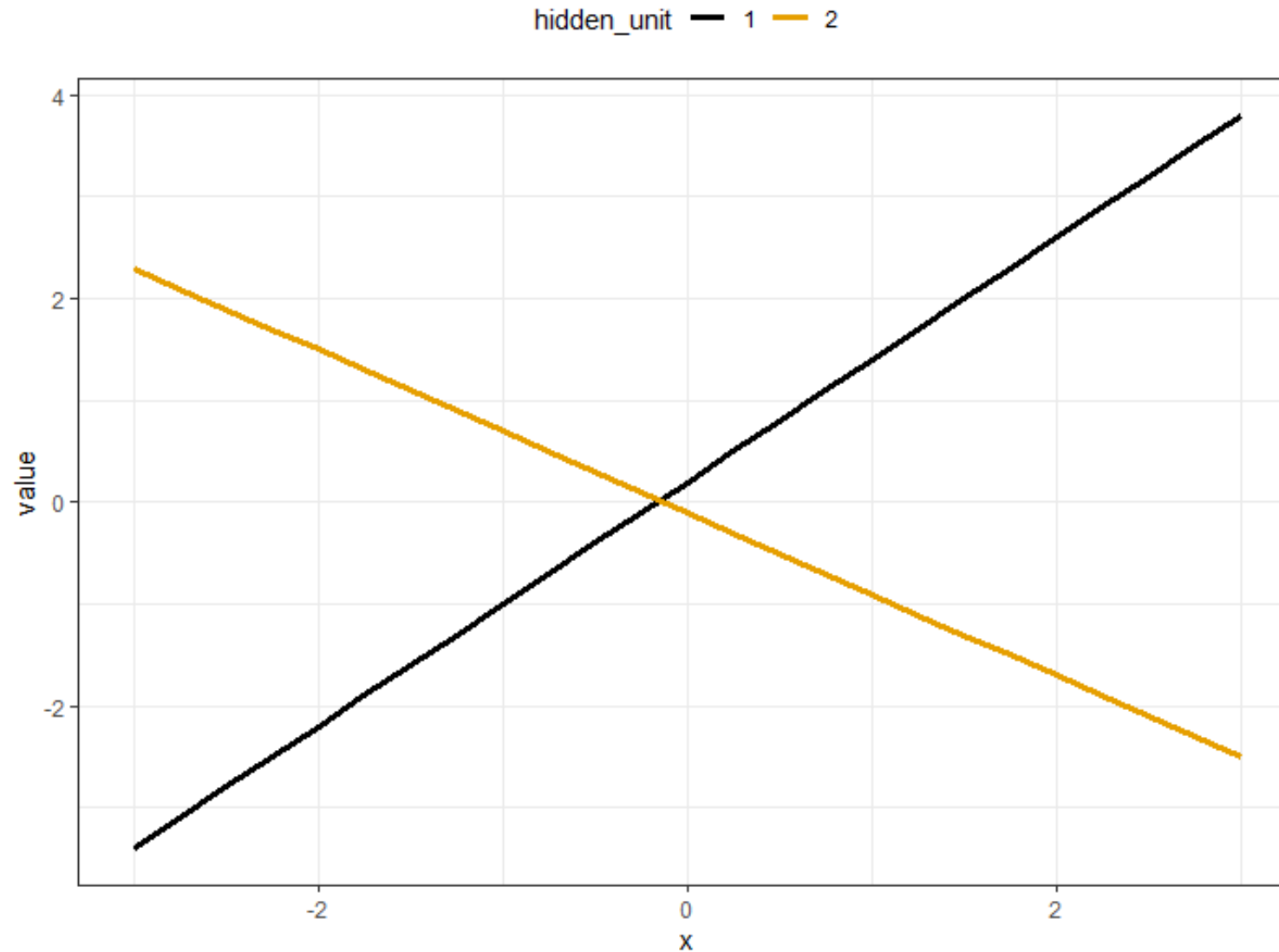
```
5 -2.2 1.5
```

```
6 -1.9 1.3
```



$$\mathbf{A} = \mathbf{XB}$$

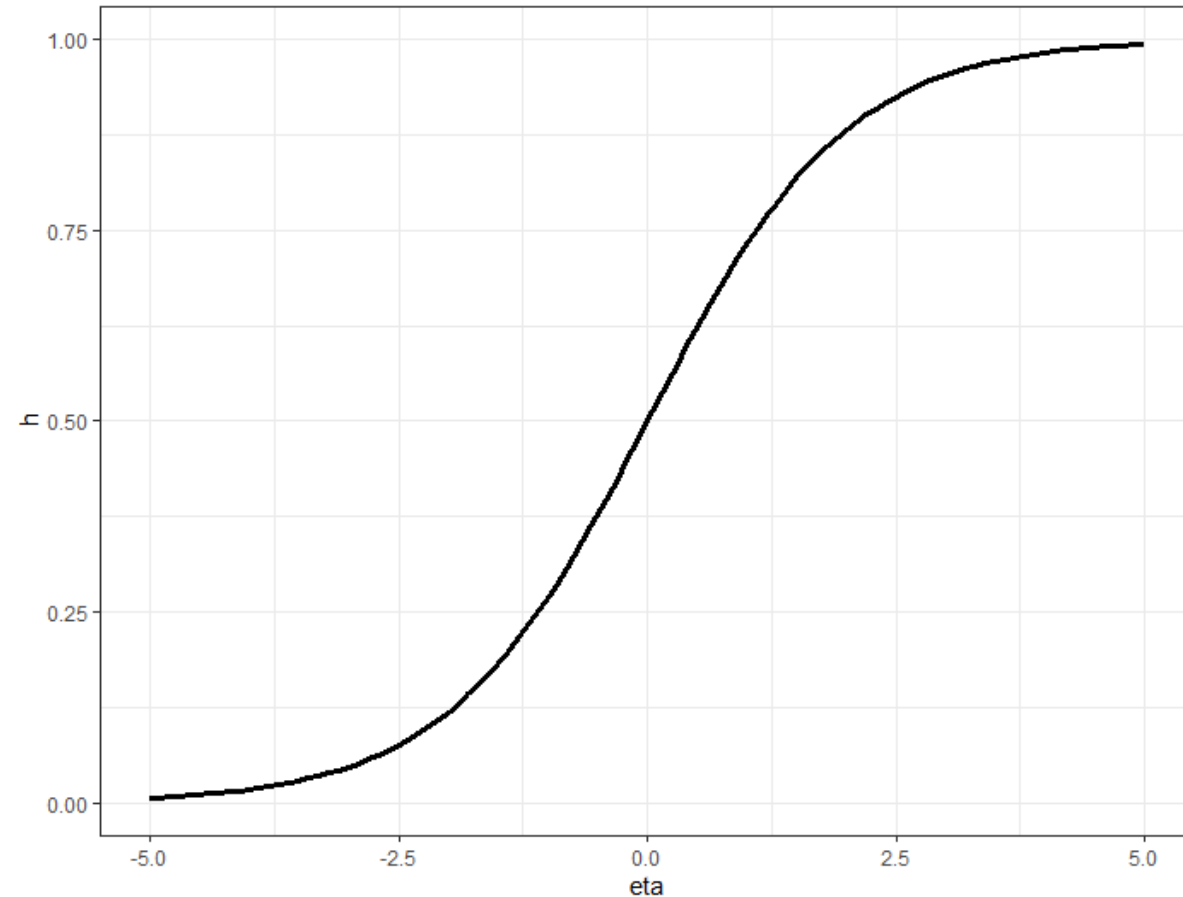
Hidden unit linear combinations



Non-linear transformation functions

- Many different functions are used in practice.
- The logistic function is a common choice.

$$h = \text{logit}^{-1}(\eta) = \frac{1}{1 + \exp(-\eta)} = \frac{\exp(\eta)}{1 + \exp(\eta)}$$

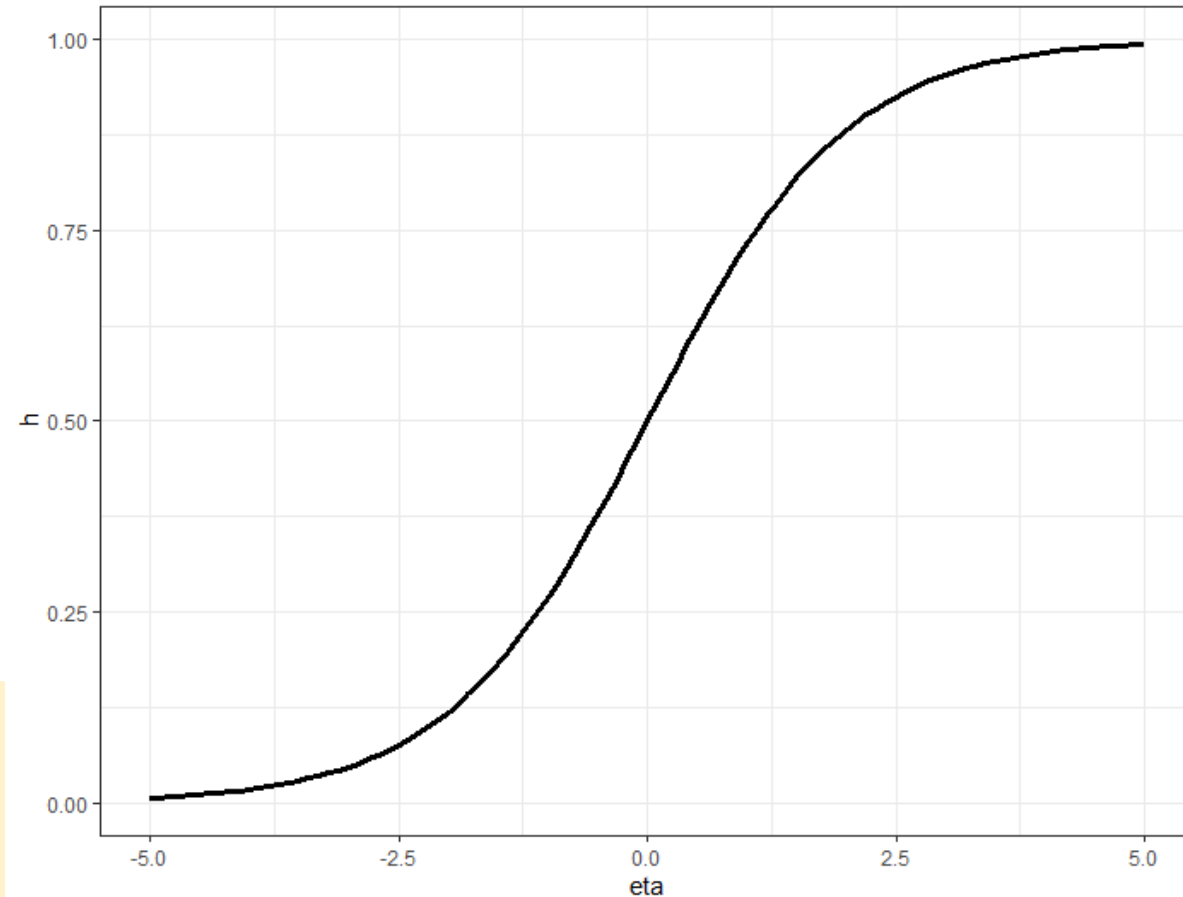


Non-linear transformation functions

- Many different functions are used in practice.
- The logistic function is a common choice.

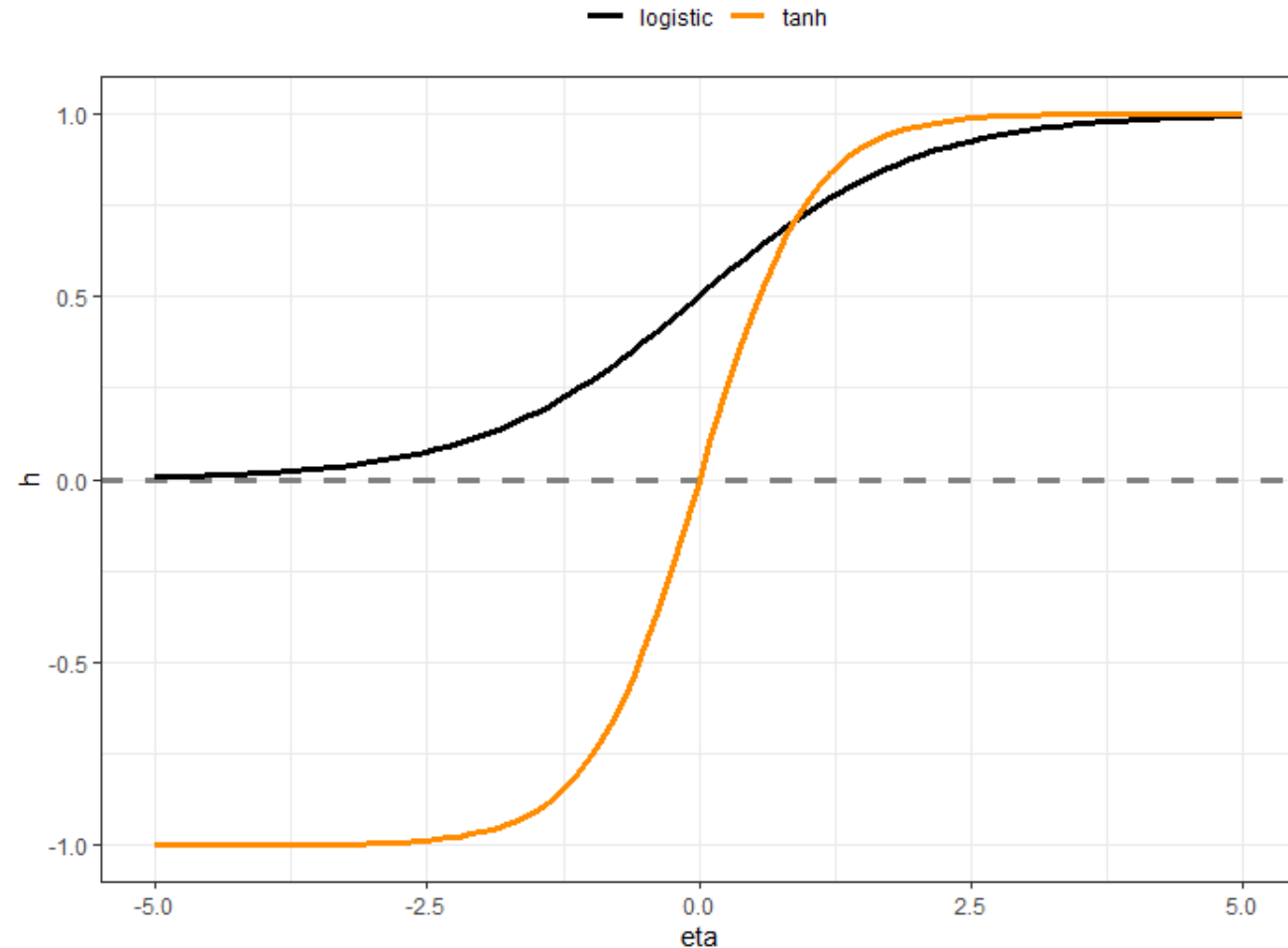
$$h = \text{logit}^{-1}(\eta) = \frac{1}{1 + \exp(-\eta)} = \frac{\exp(\eta)}{1 + \exp(\eta)}$$

If we use the logistic function as the transformation function, each hidden unit has the same functional form as **logistic regression**!



Non-linear transformation functions

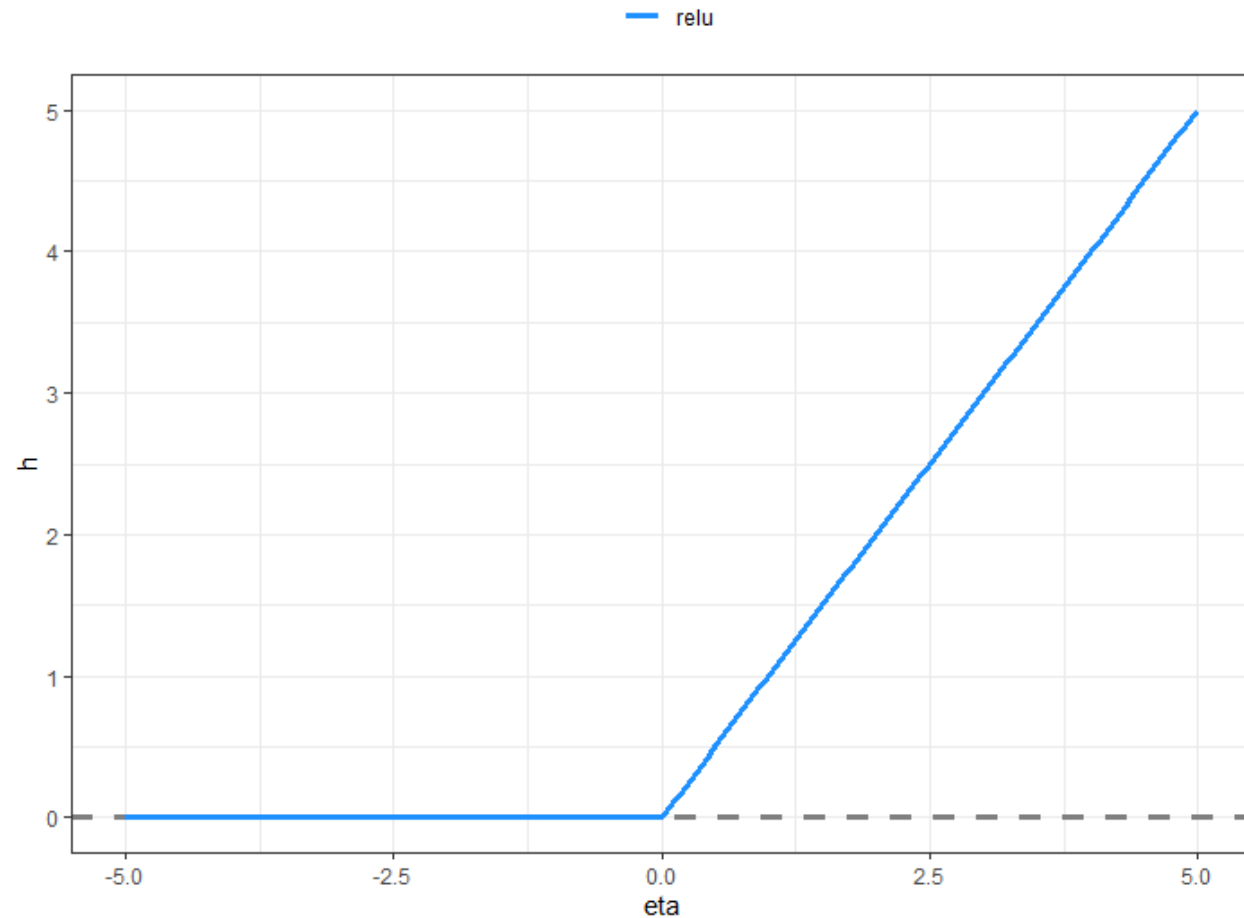
- We can also try other **S-curve** type functions.
- For example, the hyperbolic tangent, $\tanh()$, is an S-curve between -1 and +1.
- Allows negative hidden unit responses, compared to the logistic which forces positive values.



There are even some transformations that seem like linear functions!

- Rectified linear unit (ReLU).
- Sets the hidden unit response equal to the linear predictor, if the linear predictor is positive.
- If the linear predictor is negative, ReLU sets the hidden unit response to 0.

$$h = \max(0, \eta)$$



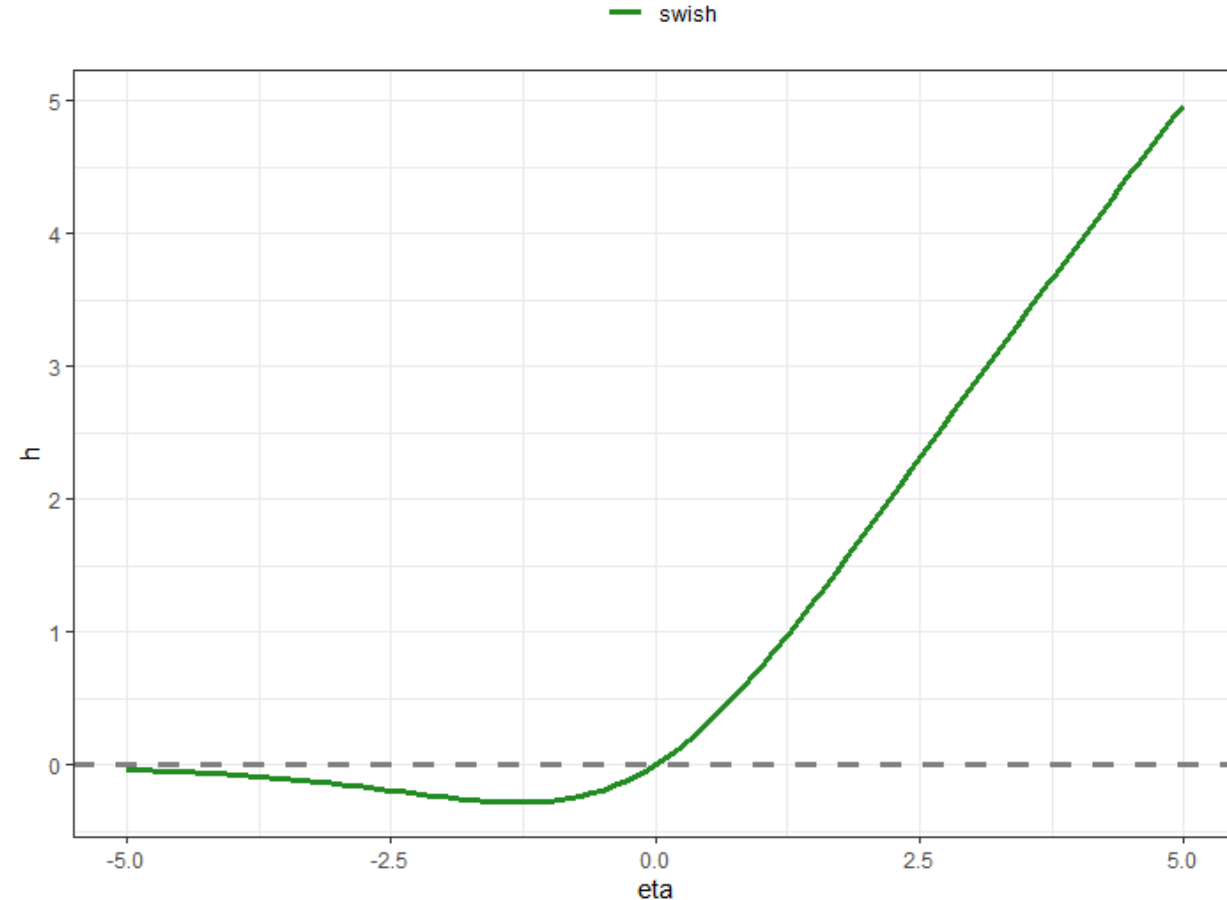
There are many alternatives, we can even combine functions together

- The swish function was created by Google.
- It is the product of a linear function and the logistic function.

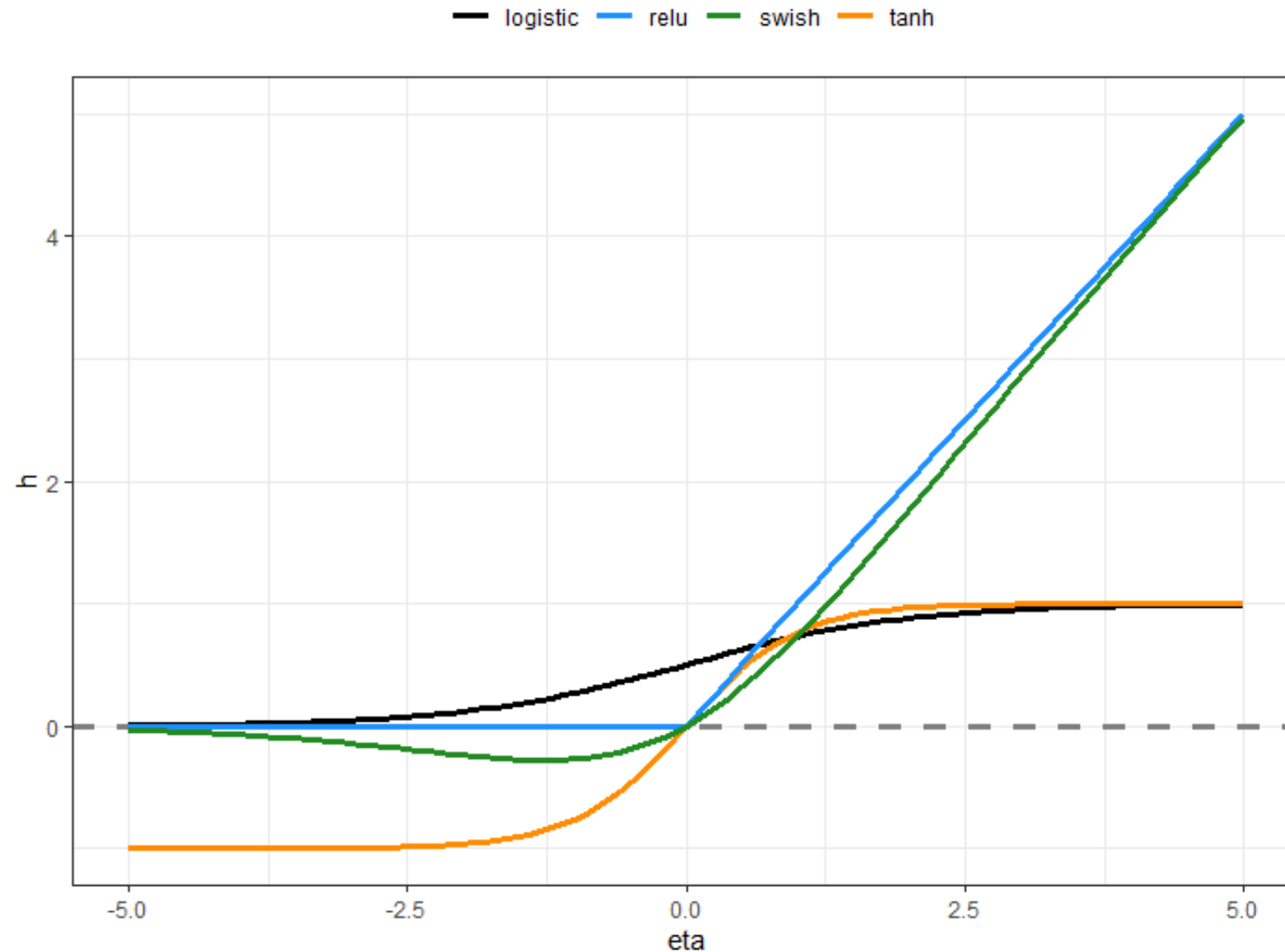
$$h = \text{Swish}(\eta) = \eta \times \text{logit}^{-1}(\eta)$$

↓
 η

$$h = \frac{\eta}{1 + \exp(-\eta)}$$



Compare these 4 non-linear transformation functions

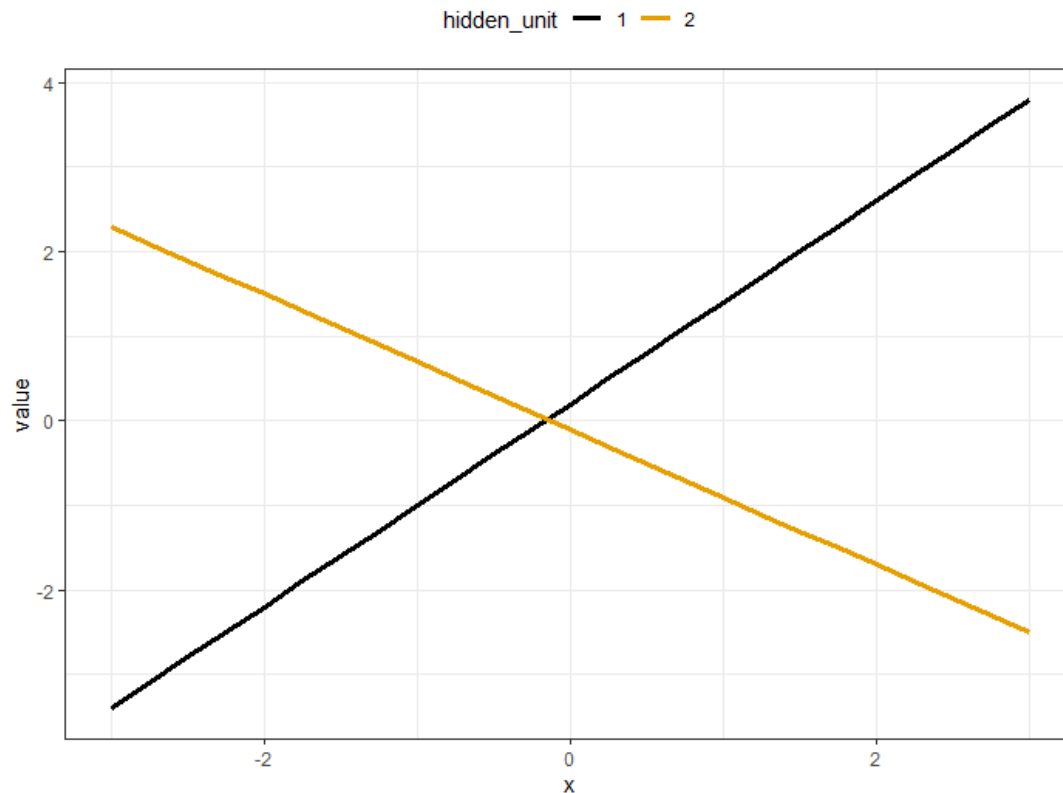


Many more options exist!

- **The transformation function must be non-linear.**
- If a linear function is used...you recover a linear model!

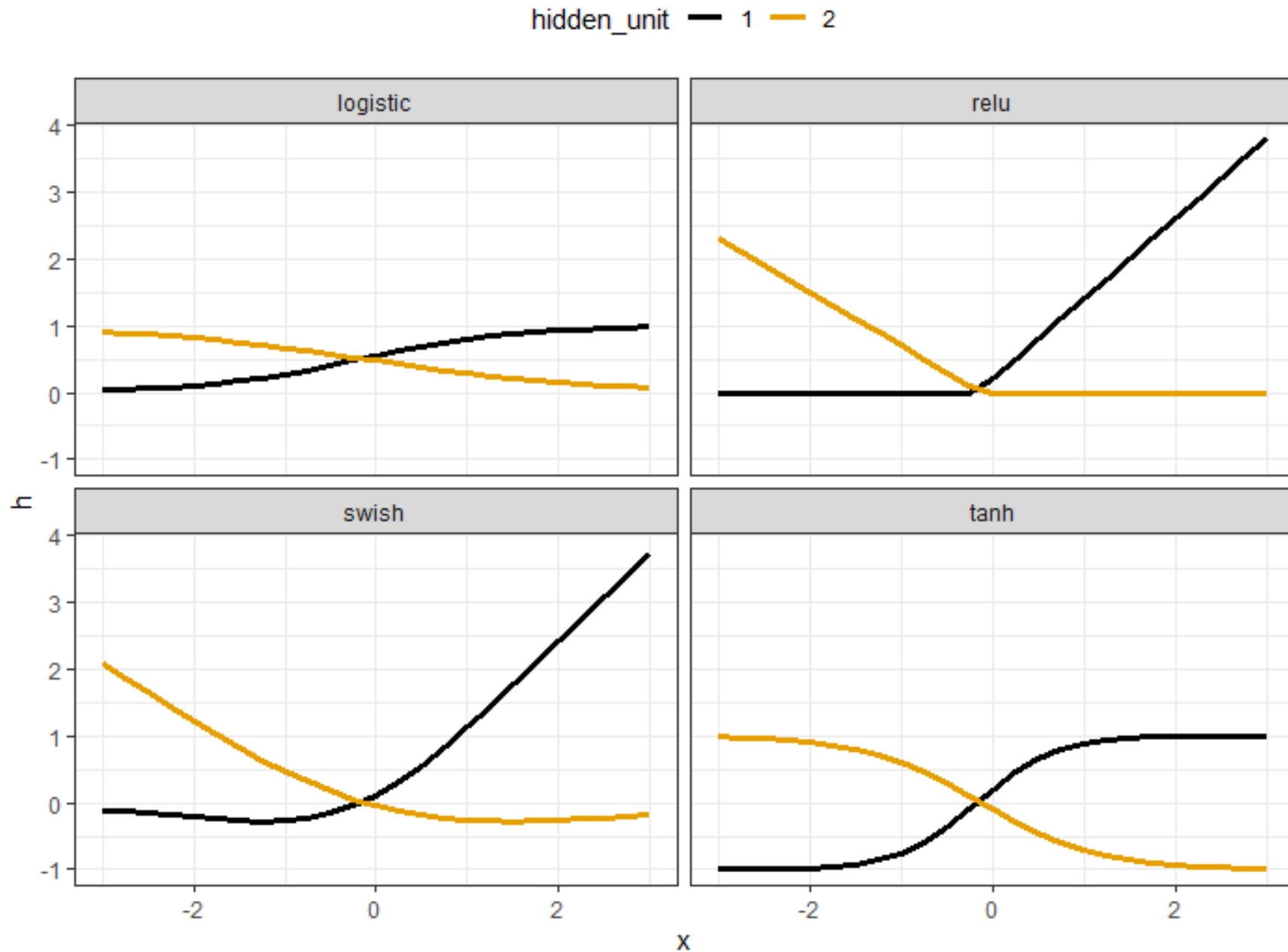
Apply these four functions to our specific example

Use the hidden unit linear combinations,
 η_k , we already calculated



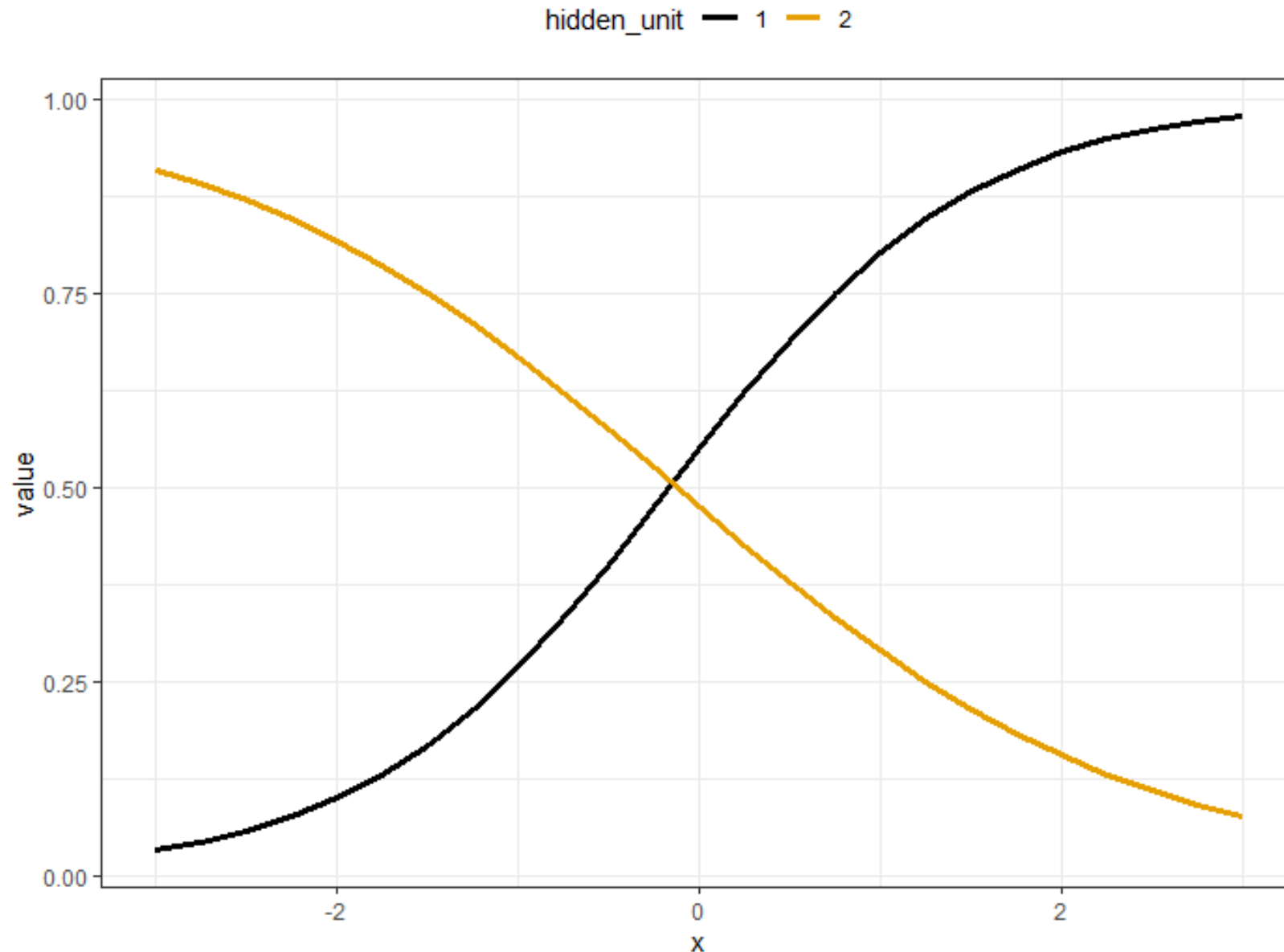
- Apply each of the 4 non-linear transformation functions we discussed in the previous slides.
- Logistic: $h = \text{logit}^{-1}(\eta)$
- hyperbolic tangent: $h = \tanh(\eta)$
- ReLU: $h = \max(0, \eta)$
- Swish: $h = \eta \times \text{logit}^{-1}(\eta)$

The hidden unit “trend” depends on the type of non-linear transformation function



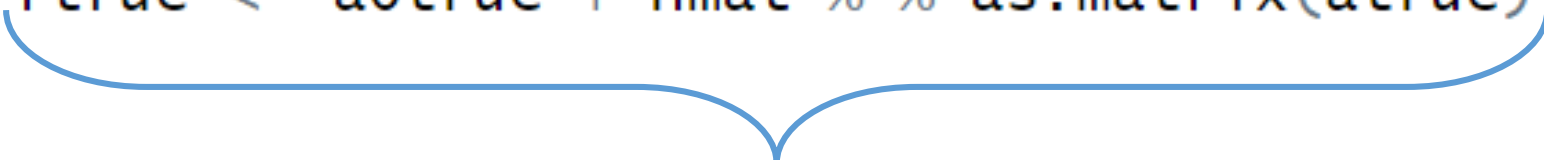
Let's use the logistic function as the non-linear function for our example

$$\mathbf{H} = g(\mathbf{A}) = g(\mathbf{XB}) = \text{logit}^{-1}(\mathbf{XB}) \longleftrightarrow \text{Hmat} \leftarrow \text{boot}::\text{inv.logit}(\text{Amat})$$



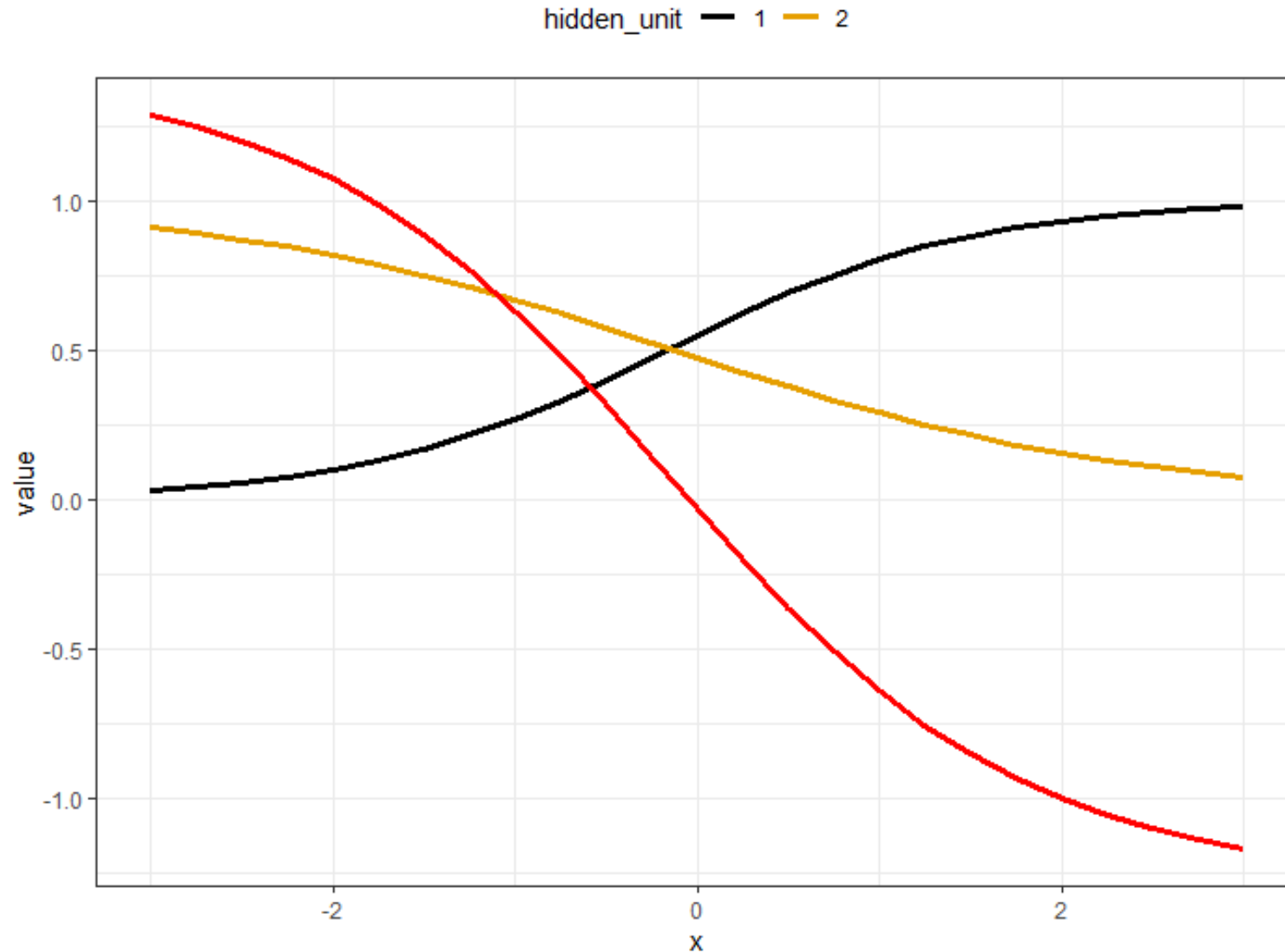
Next, we need to define the OUTPUT layer

```
### now define the true output layer parameters  
a0true <- 0.2  
atrue <- c(-1.5, 1.25)  
  
### calculate the true nnet output  
ftrue <- a0true + Hmat %*% as.matrix(atrue)
```

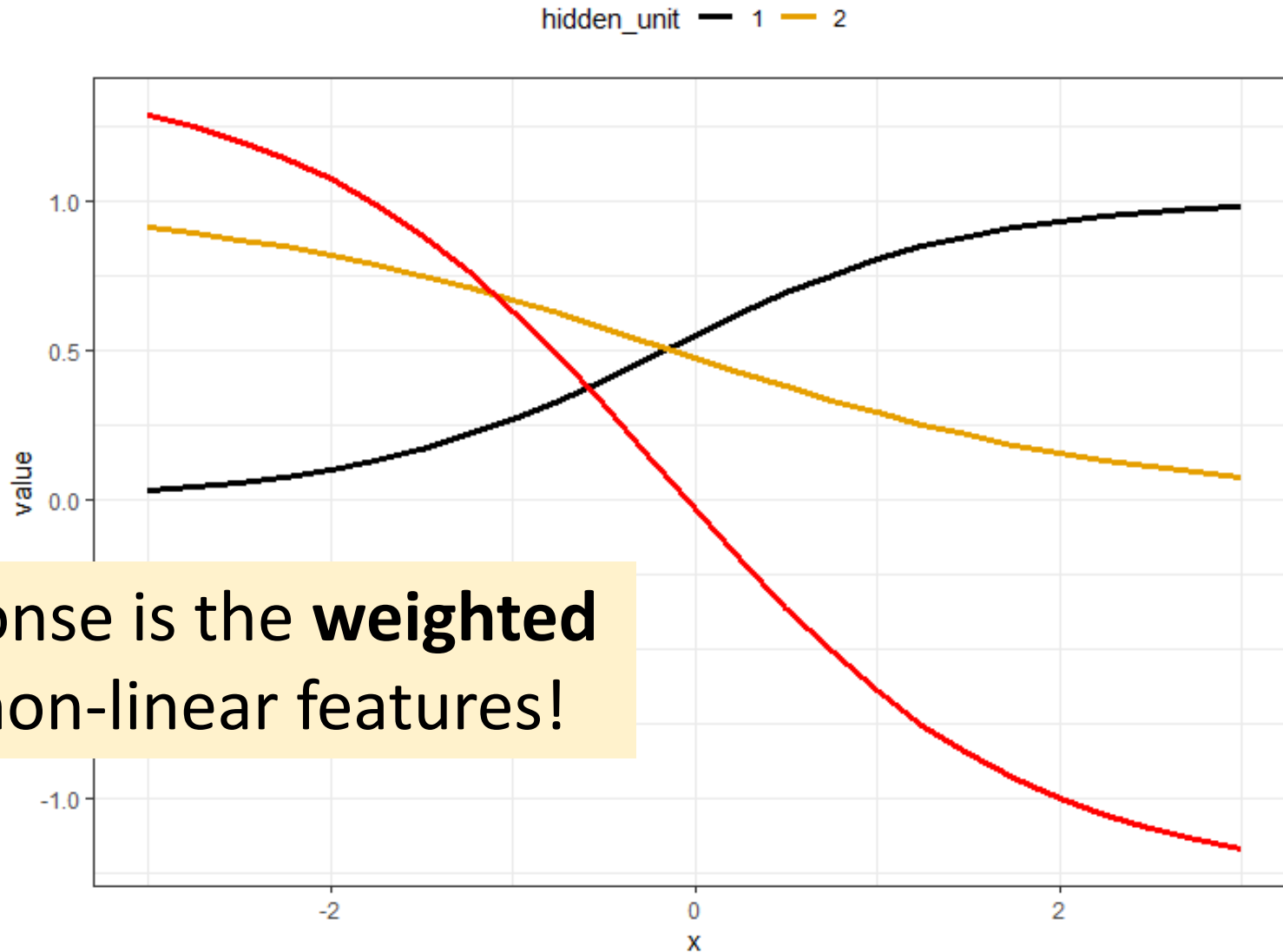


$$\mathbf{f} = \alpha_0 + \mathbf{H}\boldsymbol{\alpha} = \alpha_0 + [g(\mathbf{XB})]\boldsymbol{\alpha}$$

Neural network response compared to the hidden units



Neural network response compared to the hidden units



The response is the **weighted sum** of non-linear features!

We worked with weighted sums of non-linear features before!

- Rewrite the hidden unit response as the matrix Φ .

$$\mathbf{H} = g(\mathbf{XB}) = \Phi$$

- The neural network response is then:

$$\mathbf{f} = \alpha_0 + \Phi\alpha$$

We worked with weighted sums of non-linear features before!

- Rewrite the hidden unit response as the matrix Φ .

$$\mathbf{H} = g(\mathbf{XB}) = \Phi$$

- The neural network response is then:

$$\mathbf{f} = \alpha_0 + \boxed{\Phi \alpha}$$

A **basis** matrix multiplied by a coefficient vector!

The neural network is defining a basis for us!

- Rather than specifying the basis upfront, as with polynomials or natural splines, we will *learn* the basis.
- We specify the assumptions about how the neural network “builds up” those basis functions via the non-linear functions $g(\cdot)$.

Remember how we created the features for a natural spline model

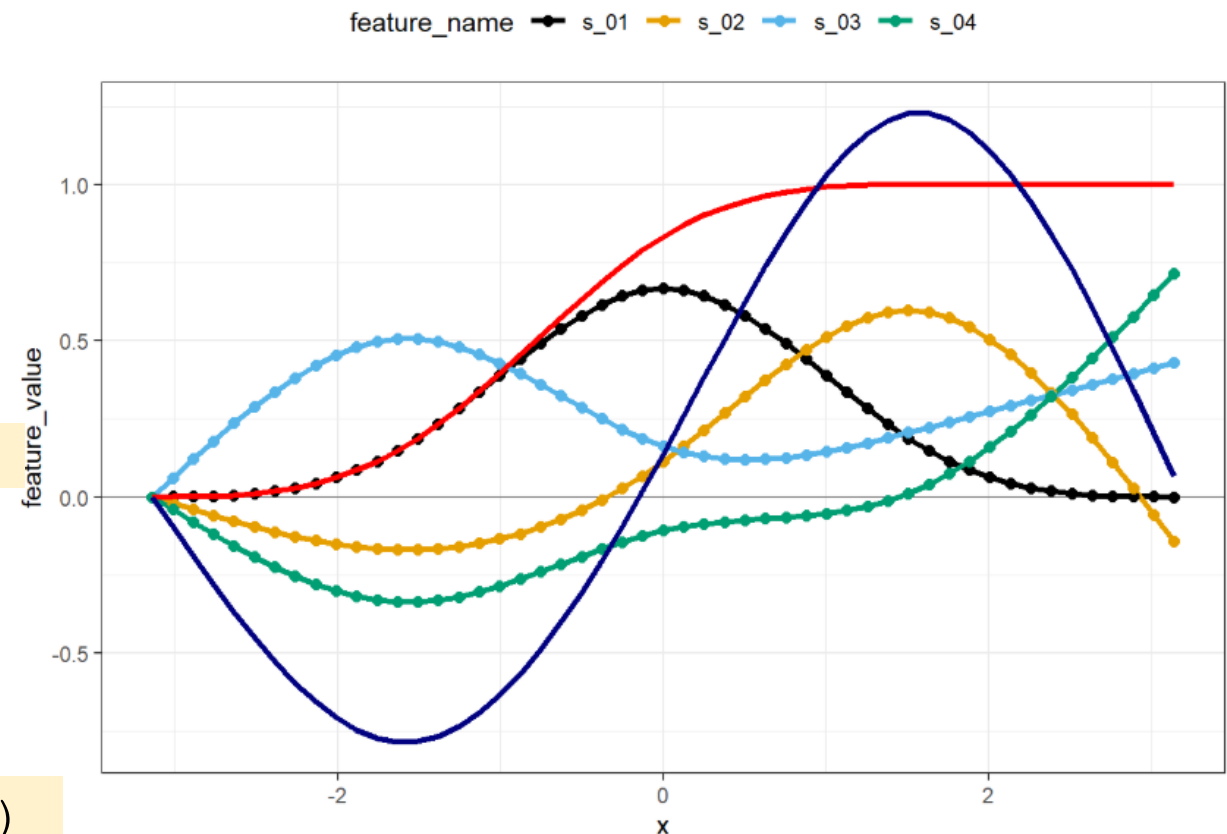
- We created the natural spline basis by just considering the input!
- The 4 features of the spline model shown to the right were assembled into a basis design matrix.

```
Xmat <- model.matrix( ~ splines::ns(x, 4), data = <data>)
```

- We learned the coefficients or weights applied to each feature with a linear model.

```
lm(y ~ splines::ns(x, 4), data = <data>)
```

4th order natural spline basis to approximate a sine wave



With neural networks, we specify “rules” through the hidden units

$$h_{n,k}(\mathbf{x}_{n,:}) = g \left(\underbrace{\sum_{d=0}^D x_{n,d} \beta_{d,k}} \right)$$

Weighted sum of all
input variables



linear models with linear relationships!
“linear combinations” $\leftrightarrow$ linear relationships

With neural networks, we specify “rules” through the hidden units

$$h_{n,k}(\mathbf{x}_{n,:}) = g \left(\sum_{d=0}^D x_{n,d} \beta_{d,k} \right)$$

Specify a non-linear, yet relatively simple function

Generalize the linear models!

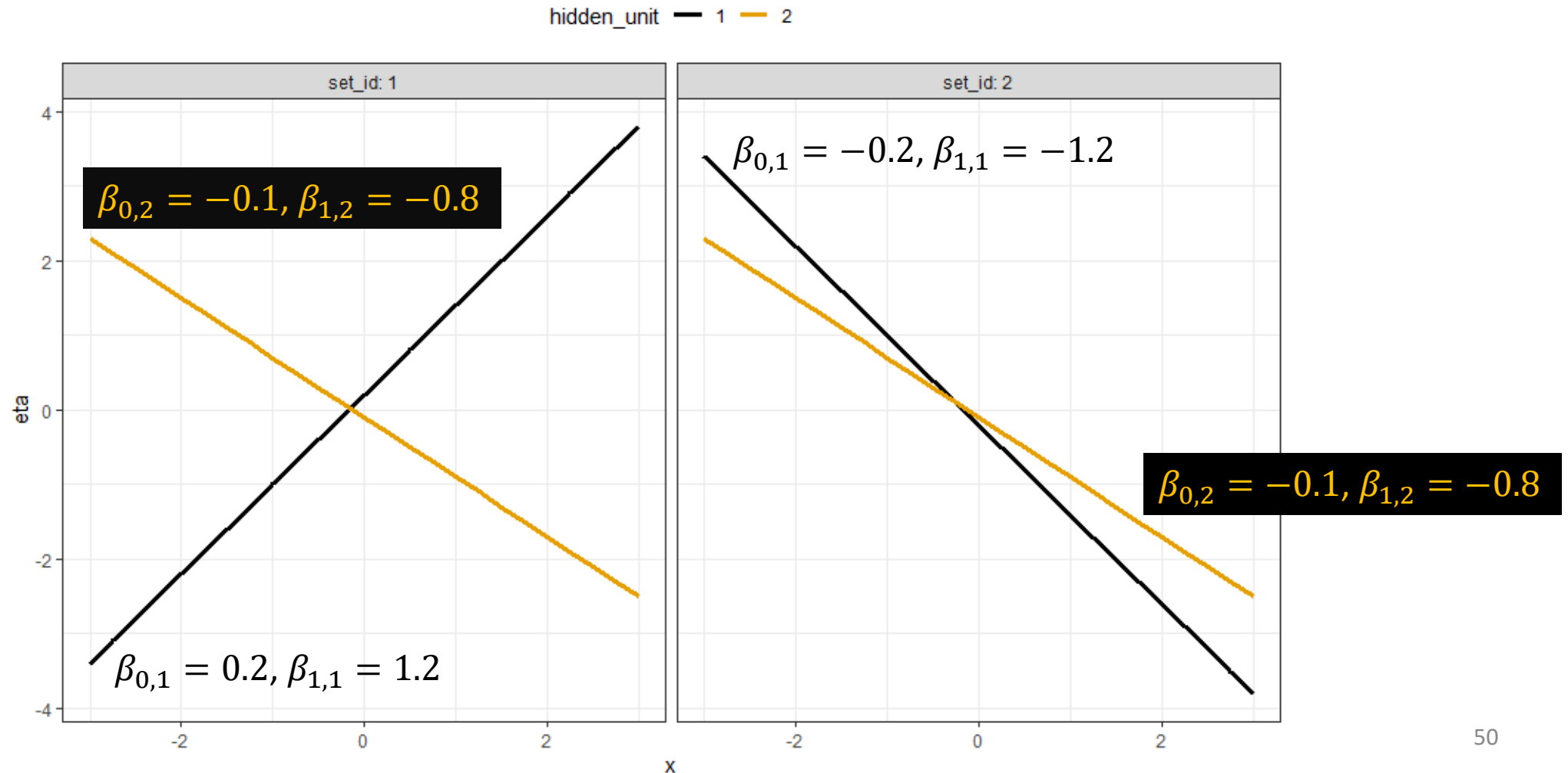
The flexibility of neural networks comes from the fact that we must LEARN the hidden unit parameters from the data!

$$h_{n,k}(\mathbf{x}_{n,:}) = g \left(\sum_{d=0}^D x_{n,d} \beta_{d,k} \right)$$

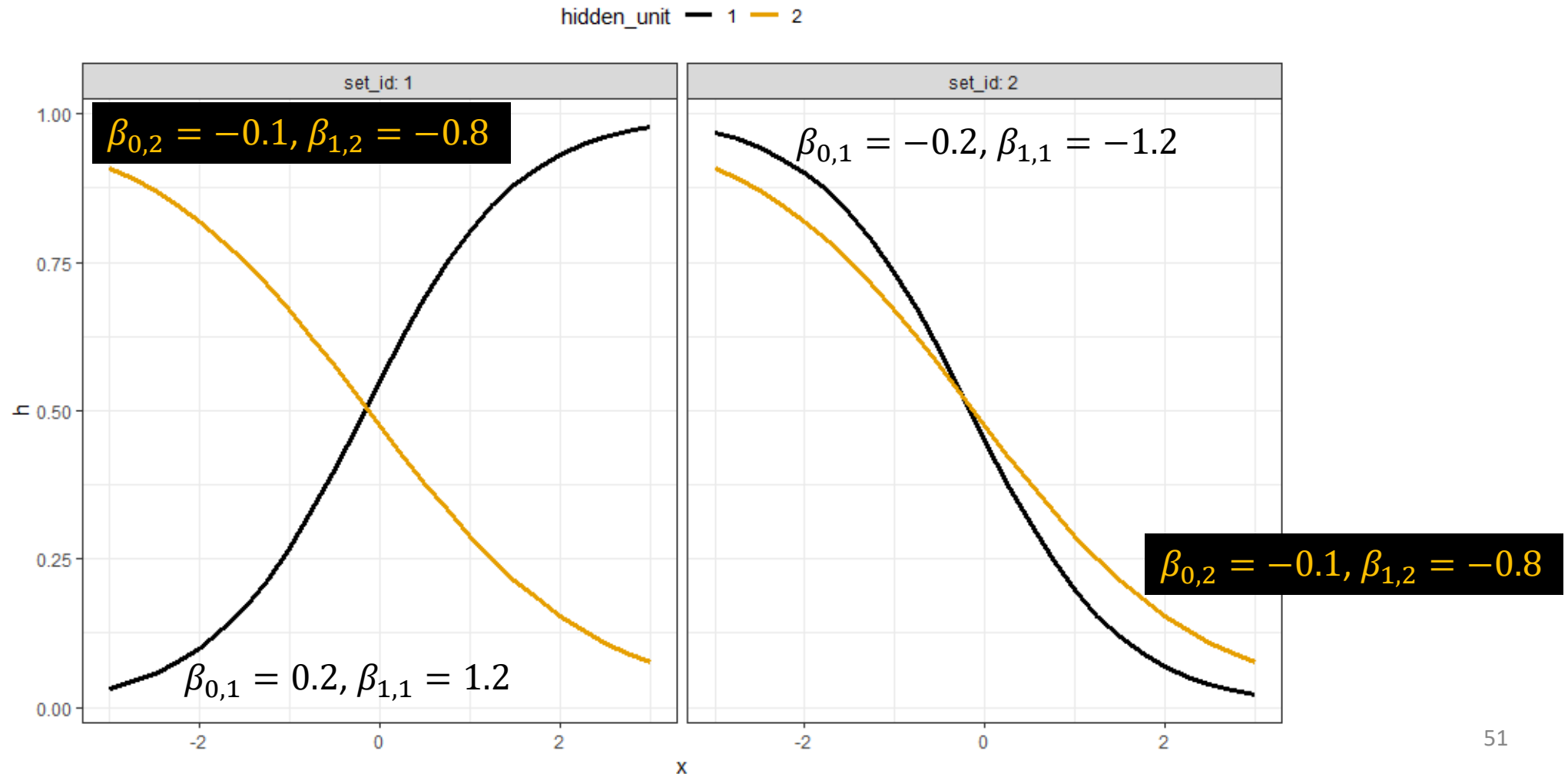
If we fixed the values of all of the β -parameters (the hidden weights) before fitting, we would have a fixed basis, like the natural splines.

Instead, we want to adapt the basis to the data!

Essentially, if we change the weights, we change the hidden unit behavior



If we change the hidden units, we change the basis functions of the model!



Fitting or training a neural network requires learning all hidden units parameters!

- The biases and weights, $\{\boldsymbol{\beta}_{k=1}, \boldsymbol{\beta}_{k=2}, \dots, \boldsymbol{\beta}_{k=H}\}$, are unknown.
- We collect inputs, $\mathbf{X}$, and responses, $\mathbf{y}$.
- Our goal is to learn the hidden coefficients, and therefore learn the hidden basis of the model!

Number of parameters

- $D + 1$ β -parameters in **EACH** hidden unit.
- Total number of hidden unit parameters:

$$H(D + 1)$$

Number of parameters

- $D + 1$ β -parameters in **EACH** hidden unit.
- Total number of hidden unit parameters:

$$H(D + 1)$$

Is this all of the unknowns?

We also need to learn the output layer parameters!

- The output layer is a weighted sum of the non-linear features:

$$\alpha_0 + [g(\mathbf{XB})]\alpha$$

- $H + 1$ α -parameters in the output layer.

- **TOTAL number of unknown parameters:**

$$H(D + 1) + H + 1$$

We must learn all of the parameters from the available data.

We also need to learn the output layer parameters!

- The output layer is a weighted sum of the non-linear features:

$$\alpha_0 + [g(\mathbf{XB})]\alpha$$

- $H + 1$ α -parameters in the output layer.

- **TOTAL number of unknown parameters:**

$$H(D + 1) + H + 1$$

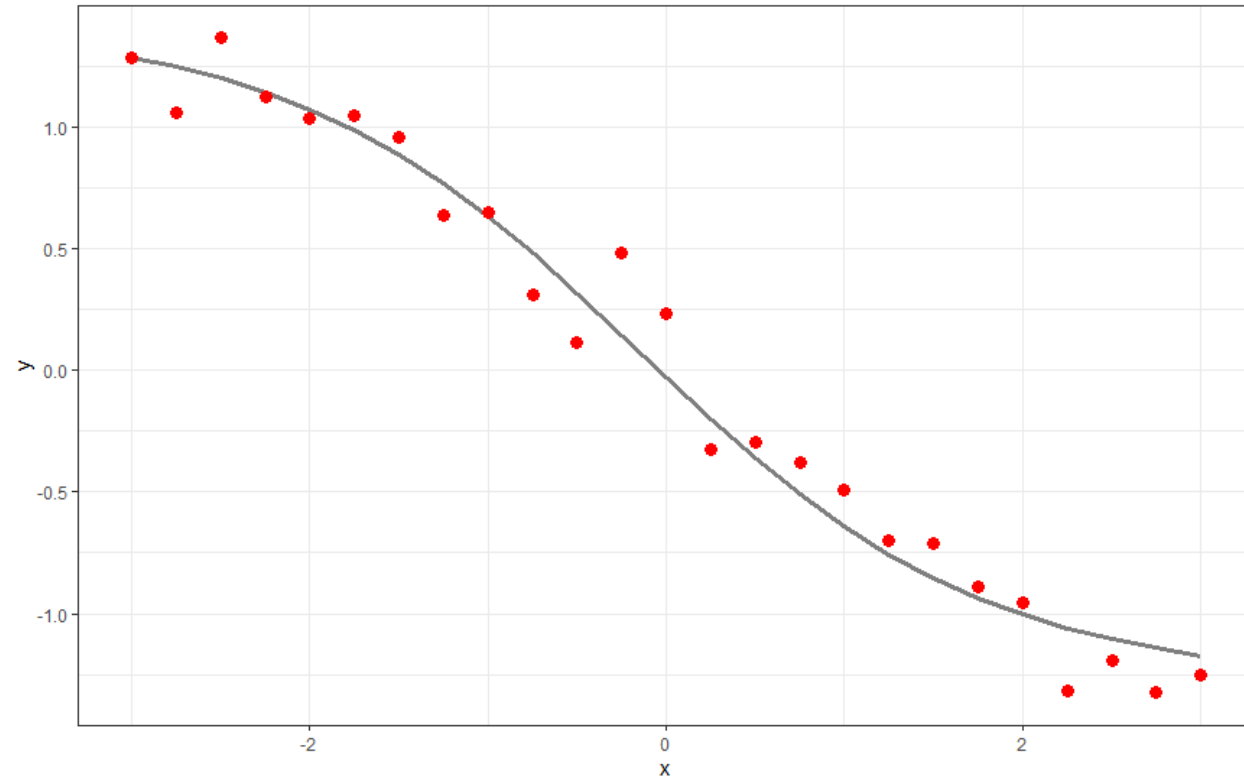
Just for a single
hidden layer
neural network!

We must learn all of the parameters from the available data.

Let's practice fitting neural networks by creating some noisy observations from our toy problem

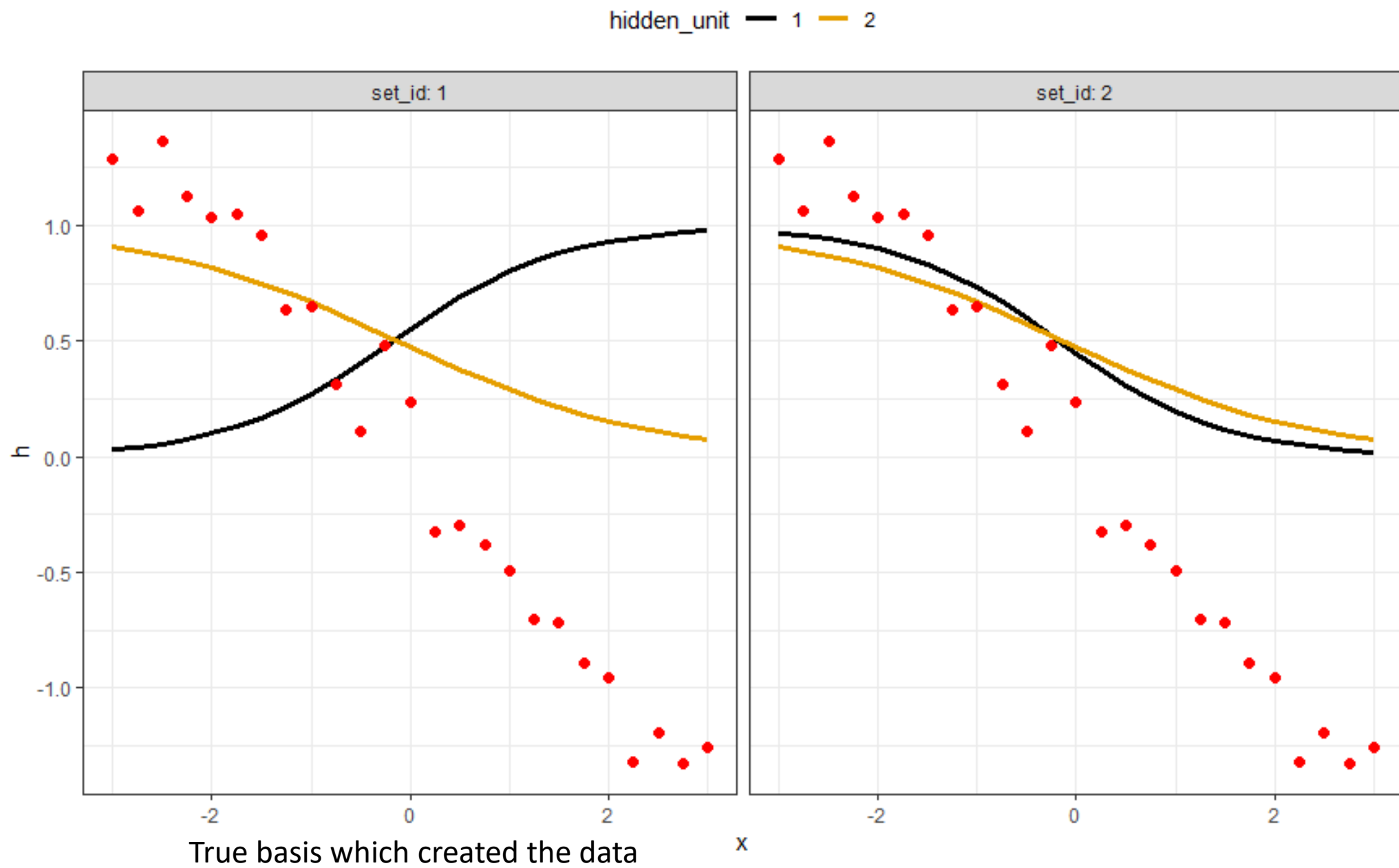
```
### set the noise to a low value
sigma_true <- 0.15

### generate noisy observations and package
### the input with the noisy response
set.seed(1131)
demo_df <- tibble::tibble(x = x) %>%
  mutate(f = ftrue,
         y = rnorm(n = n(), mean = f, sd = sigma_true))
```

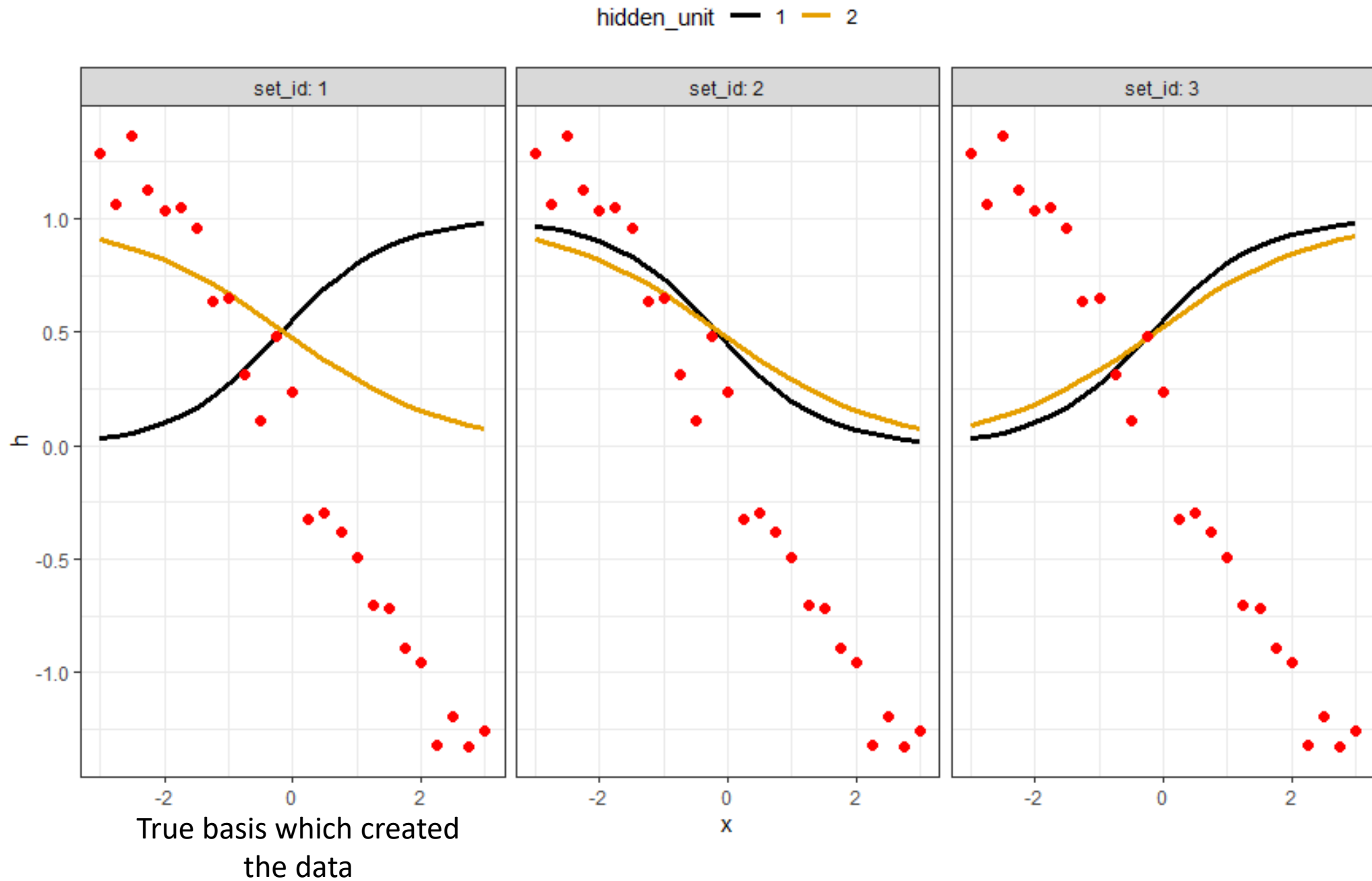


Our goal is to learn all hidden unit parameters, β 's, and output layer parameters, α 's.

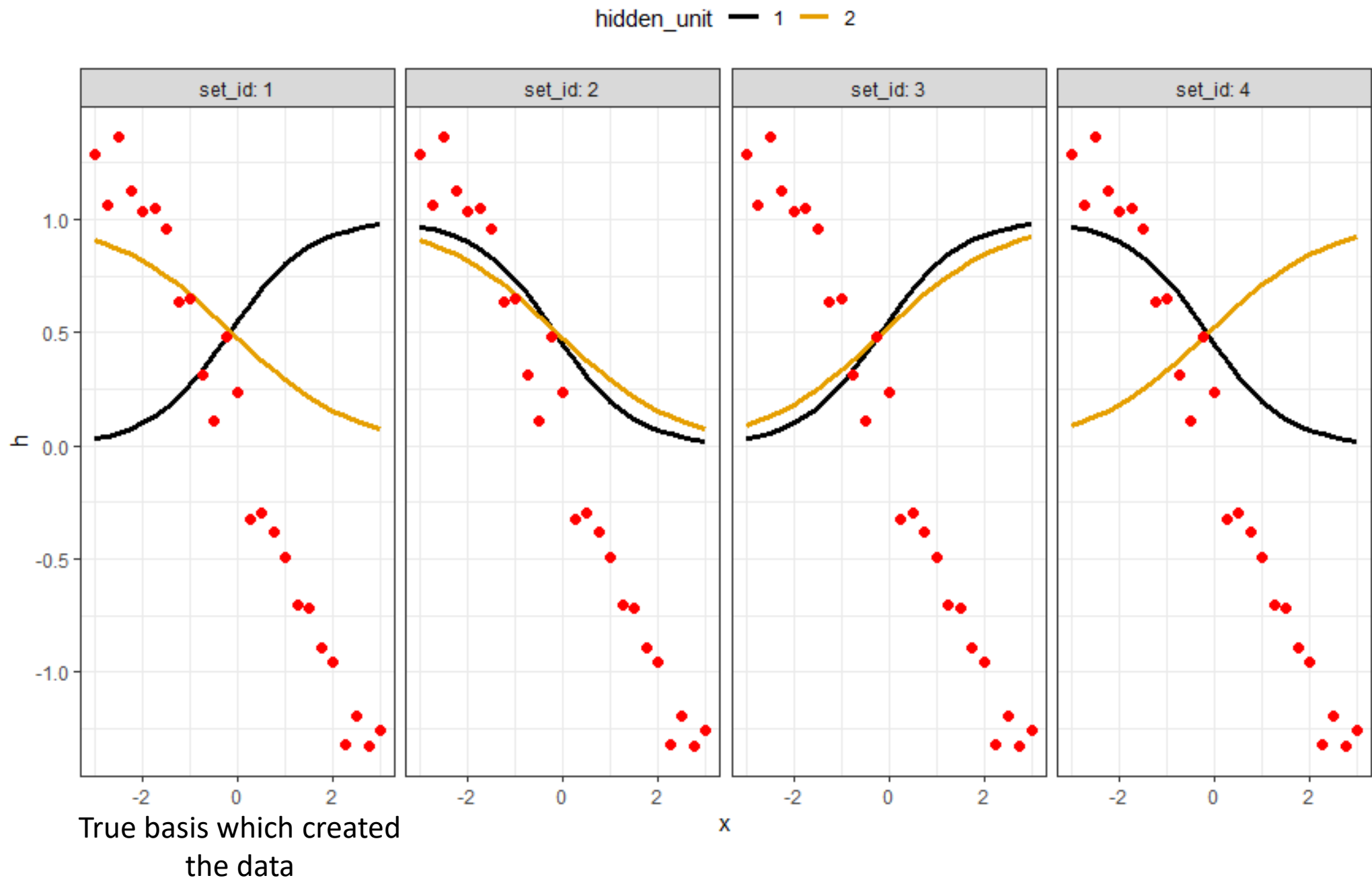
- We will fit the model such that the hidden units combine to approximate the output.
- Consider the 2 sets of hidden unit parameters we already visualized.
- The first set created the data, but how does the second set compare?



What if we tried a third set of parameters?



Or a fourth set of parameters...



Fitting the neural network requires specifying an objective/utility to maximize, or a cost/loss/error function to minimize

- Neural networks for regression problems are typically trained by minimizing the sum of squared errors (SSE):

$$\underset{\beta_{k=1}, \beta_{k=2}, \dots, \beta_{k=H}, \alpha_0, \alpha}{\operatorname{argmin}} \left(\sum_{n=1}^N ((y_n - f_n)^2) \right)$$

Fitting the neural network requires specifying an objective/utility to maximize, or a cost/loss/error function to minimize

- Neural networks for regression problems are typically trained by minimizing the sum of squared errors (SSE):

$$\underset{\beta_{k=1}, \beta_{k=2}, \dots, \beta_{k=H}, \alpha_0, \alpha}{\operatorname{argmin}} \left(\sum_{n=1}^N ((y_n - f_n)^2) \right)$$

The SSE has the same functional form as
a Gaussian log-likelihood!

Minimizing the SSE is equivalent to maximizing the Gaussian likelihood!

$$\sum_{n=1}^N ((y_n - f_n)^2) \leftrightarrow \sum_{n=1}^N (\log[\text{normal}(y_n \mid f_n, \sigma)])$$



$$\log[\text{normal}(y_n \mid f_n, \sigma)] \propto -\frac{1}{2} \log[\sigma^2] - \frac{1}{2\sigma^2} (y_n - f_n)^2$$

The neural network response, f_n , is treated as the **MEAN** of the n -th observed response, y_n .

The neural network response, f_n , is analogous to the linear model mean trend μ_n !

- With a linear model, the OLS estimator gives the parameter values that maximize the likelihood.
- However with a neural network, the response is NON-LINEARLY related to the hidden unit parameters.
- **We cannot find a closed form solution to the unknown parameters!**
- We must iterate, like we did with logistic regression, to find the parameter estimates.

Previously, we would start out by visualizing the log-likelihood or log-posterior surface with respect the unknowns...

- However, even with this simple neural network of 2 hidden units and a single input it is very difficult to visualize the surface!
- The total number of unknown parameters:

$$H(D + 1) + H + 1 = 2 \times 2 + 2 + 1 = 7$$

Let's focus on the behavior of the hidden units, since the dimensionality is rather high

- Before going through the detailed math, let's fit some models and see what happens.
- We will use existing libraries within $\mathbb{R}$ to fit a neural network to our simple toy demo data set.
- There are many different neural network packages we can use.
- Some are native to $\mathbb{R}$, some are APIs to external libraries.

To get a sense for how many methods exist...look at Available Models from the `caret` package documentation:

<http://topepo.github.io/caret/available-models.html>

Search for all models associated with “neural” yields 12 results.

Search:

Model	<i>method</i> Value	Type	Libraries	Tuning Parameters
Bayesian Regularized Neural Networks	brnn	Regression	brnn	neurons
Dynamic Evolving Neural-Fuzzy Inference System	DENFIS	Regression	frbs	Dthr, max.iter
Hybrid Neural Fuzzy Inference System	HYFIS	Regression	frbs	num.labels, max.iter
Model Averaged Neural Network	avNNet	Classification, Regression	nnet	size, decay, bag
Monotone Multi-Layer Perceptron Neural Network	monmlp	Classification, Regression	monmlp	hidden1, n.ensemble
Neural Network	mxnet	Classification, Regression	mxnet	layer1, layer2, layer3, learning.rate, momentum, dropout, activation
Neural Network	mxnetAdam	Classification, Regression	mxnet	layer1, layer2, layer3, dropout, beta1, beta2, learningrate, activation
Neural Network	neuralnet	Regression	neuralnet	layer1, layer2, layer3
Neural Network	nnet	Classification, Regression	nnet	size, decay
Neural Networks with Feature Extraction	pcaNNet	Classification, Regression	nnet	size, decay
Quantile Regression Neural Network	qrnn	Regression	qrnn	n.hidden, penalty, bag
Stacked AutoEncoder Deep Neural Network	dnn	Classification, Regression	deepnet	layer1, layer2, layer3, hidden_dropout, visible_dropout

Showing 1 to 12 of 12 entries (filtered from 238 total entries)

To get a sense for how many methods exist...look at Available Models from the `caret` package documentation:

<http://topepo.github.io/caret/available-models.html>

Search for all models associated with “percep” yields 10 results.

Includes neural networks associated with the Keras API.

Search: percep					
Model	method	Value	Type	Libraries	Tuning Parameters
Monotone Multi-Layer Perceptron Neural Network	monmlp		Classification, Regression	monmlp	hidden1, n.ensemble
Multi-Layer Perceptron	mlp		Classification, Regression	RSNNS	size
Multi-Layer Perceptron	mlpWeightDecay		Classification, Regression	RSNNS	size, decay
Multi-Layer Perceptron, multiple layers	mlpWeightDecayML		Classification, Regression	RSNNS	layer1, layer2, layer3, decay
Multi-Layer Perceptron, with multiple layers	mlpML		Classification, Regression	RSNNS	layer1, layer2, layer3
Multilayer Perceptron Network by Stochastic Gradient Descent	mlpSGD		Classification, Regression	FCNN4R, plyr	size, l2reg, lambda, learn_rate, momentum, gamma, minibatchsz, repeats
Multilayer Perceptron Network with Dropout	mlpKerasDropout		Classification, Regression	keras	size, dropout, batch_size, lr, rho, decay, activation
Multilayer Perceptron Network with Dropout	mlpKerasDropoutCost		Classification	keras	size, dropout, batch_size, lr, rho, decay, cost, activation
Multilayer Perceptron Network with Weight Decay	mlpKerasDecay		Classification, Regression	keras	size, lambda, batch_size, lr, rho, decay, activation
Multilayer Perceptron Network with Weight Decay	mlpKerasDecayCost		Classification	keras	size, lambda, batch_size, lr, rho, decay, cost, activation

Showing 1 to 10 of 10 entries (filtered from 238 total entries)

We will not focus on the particulars of specific libraries like Keras, h2o, mxnet, TensorFlow, Torch

- Please see Ch. 13 in the [Hands-on Machine Learning with R](#) book for details on working with deep neural networks in R via Keras or h2o.
- For specific information on the R API to Keras, please see the following link from RStudio: <https://keras.rstudio.com/>
- For more information on mxnet: <https://mxnet.apache.org/>
- RStudio AI blog has a series of articles on using Torch in R:
 - [Introduction](#), [working with torch tensors](#), [torch optimizers](#)

For this simple demo, let's use the `neuralnet` package

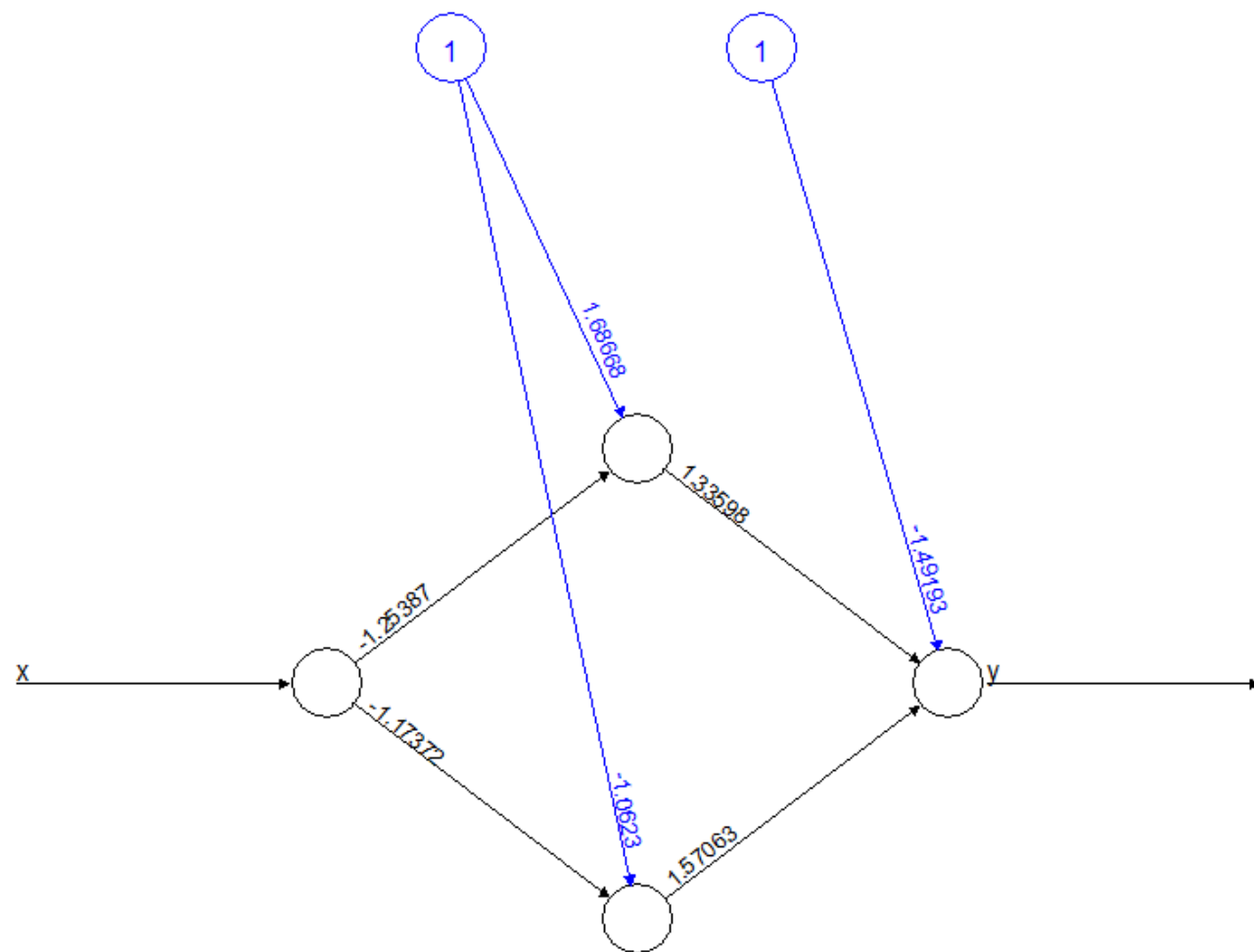
- It has a simple interface and provides basic plotting functionality.
- We will train 3 different neural networks. Each neural network will use 2 hidden units. We will use the default logistic function as the non-linear transformation (activation) function.
- Each model will be fit by minimizing the SSE (the default setting).
- **However, each model will use a different initial guess.**

```
library(neuralnet)

### first attempt
set.seed(8131)
mod1_a <- neuralnet(y ~ x,
                    data = demo_df,
                    hidden = 2,
                    err.fct = "sse",
                    act.fct = "logistic",
                    linear.output = TRUE)

### second attempt
set.seed(91123)
mod1_b <- neuralnet(y ~ x,
                    data = demo_df,
                    hidden = 2,
                    err.fct = "sse",
                    act.fct = "logistic",
                    linear.output = TRUE)

### third attempt
set.seed(5413)
mod1_c <- neuralnet(y ~ x,
                    data = demo_df,
                    hidden = 2,
                    err.fct = "sse",
                    act.fct = "logistic",
                    linear.output = TRUE)
```



Error: 0.200796 Steps: 208

```
### look at the neural network diagram  
plot(mod1_a, rep = "best")
```


Parameters are stored in the `weights` field within the returned model object

```
> mod1_a$weights
[[1]]
[[1]][[1]]
      [,1]      [,2]
[1,]  1.686682 -1.062303
[2,] -1.253870 -1.173721

[[1]][[2]]
      [,1]
[1,] -1.491932
[2,]  1.335982
[3,]  1.570633
```

```
> mod1_b$weights
[[1]]
[[1]][[1]]
      [,1]      [,2]
[1,] -0.07120246 -0.4755229
[2,] -0.78992667  0.7335158

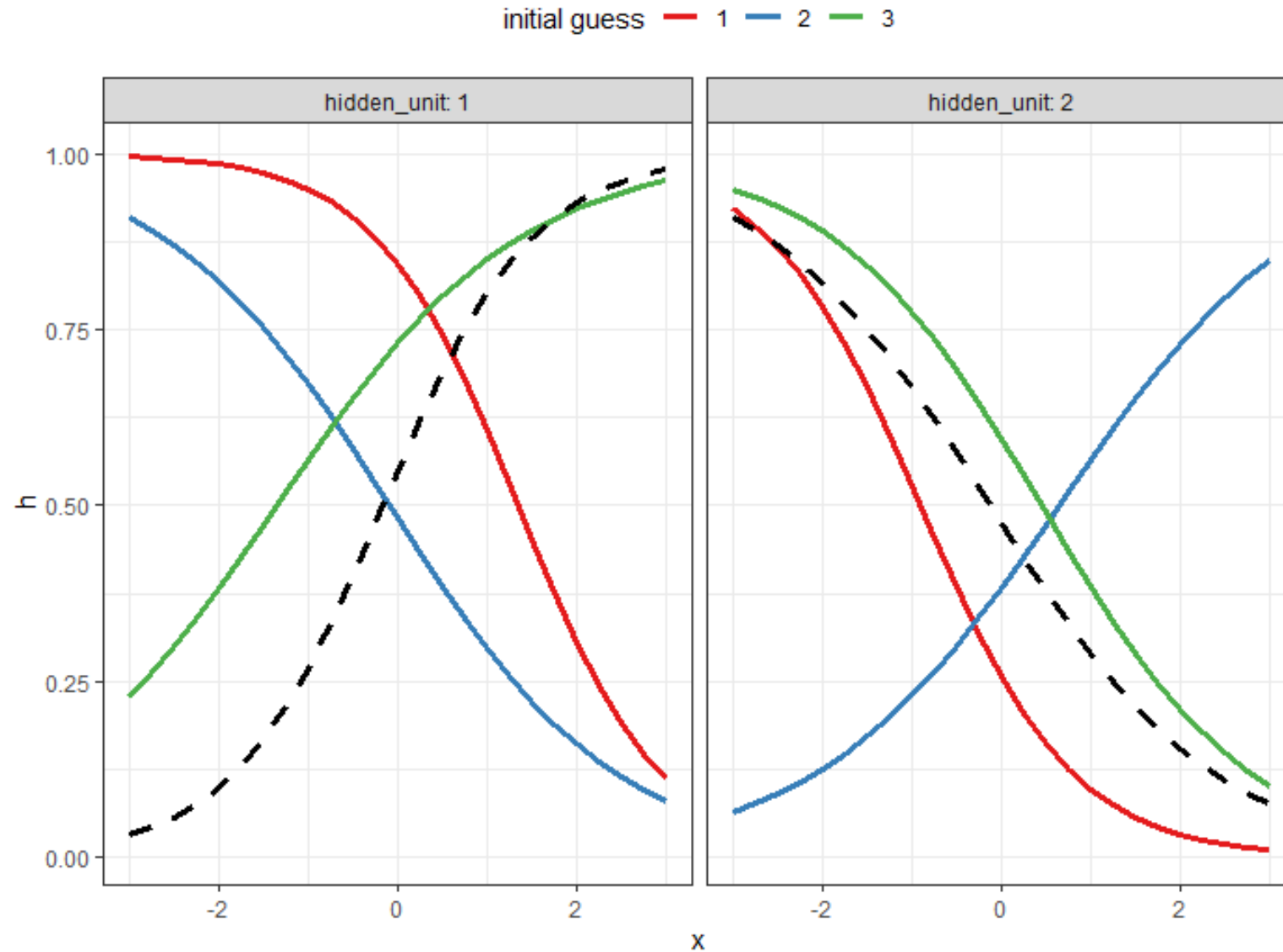
[[1]][[2]]
      [,1]
[1,] -0.4974662
[2,]  2.0596867
[3,] -1.1749833
```

```
> mod1_c$weights
[[1]]
[[1]][[1]]
      [,1]      [,2]
[1,]  0.9980648  0.3845942
[2,]  0.7370192 -0.8546132

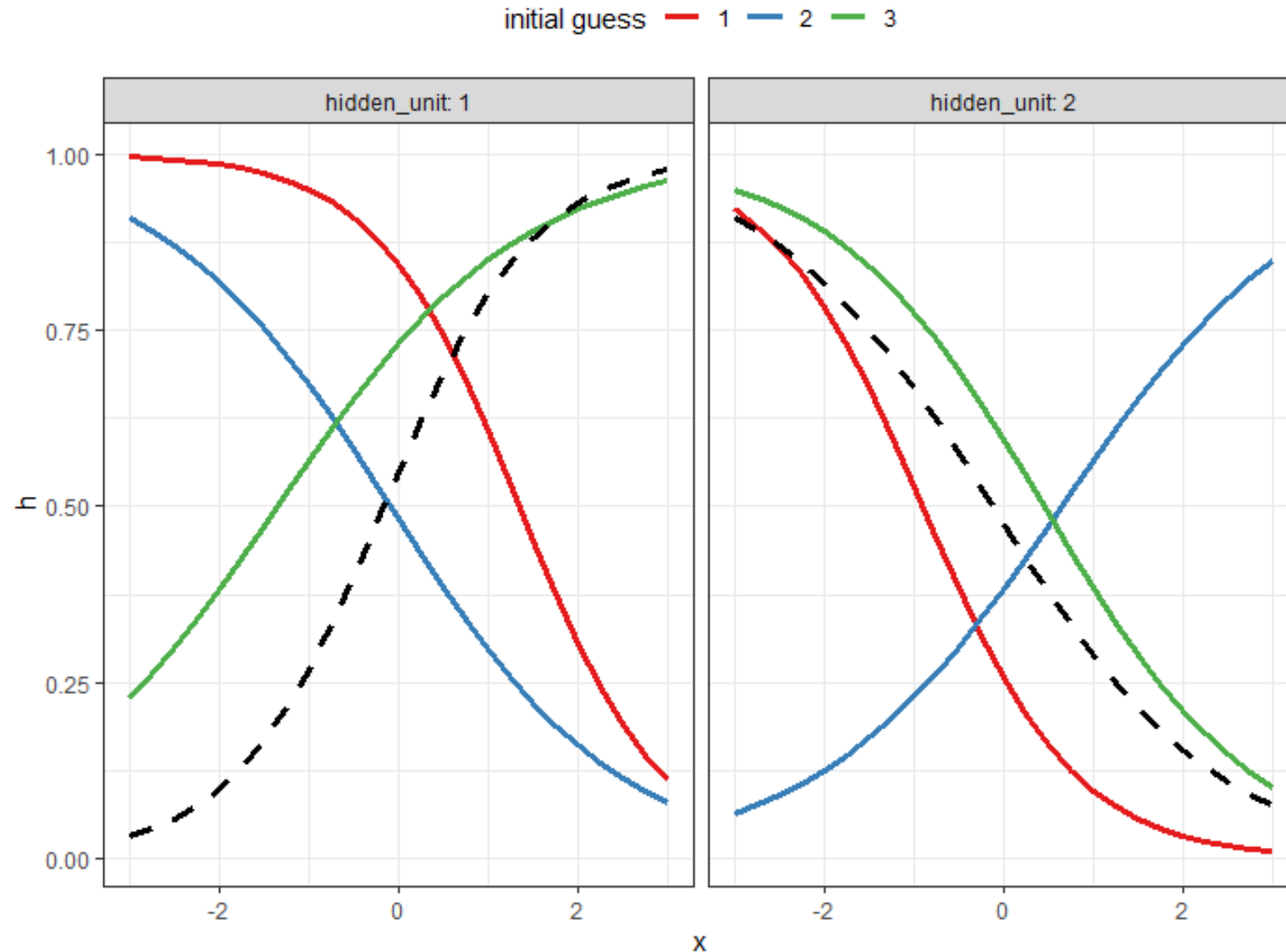
[[1]][[2]]
      [,1]
[1,] -0.7992029
[2,] -0.7961137
[3,]  2.4058118
```

The hidden unit parameters and output layer parameters are different!!!!

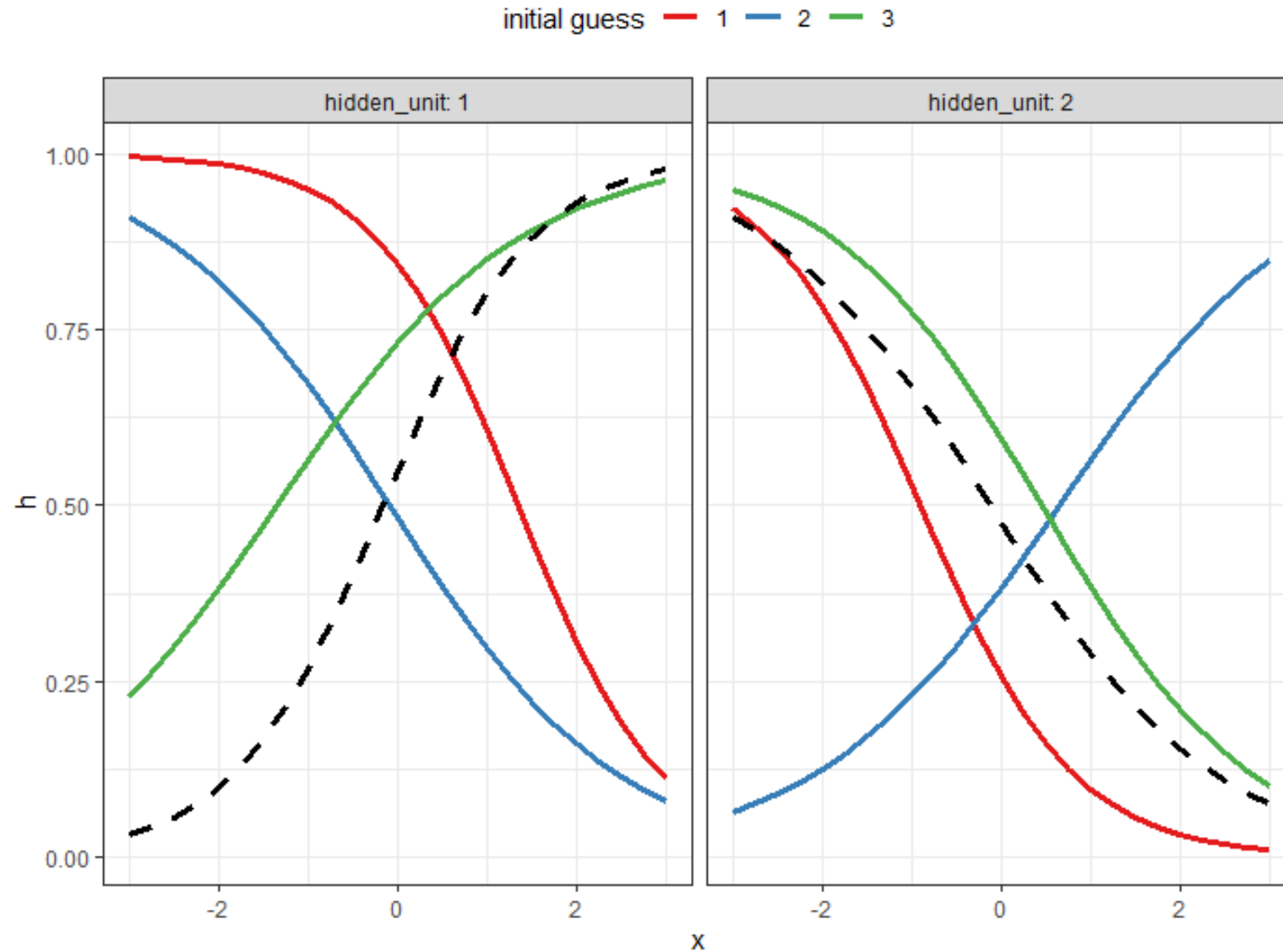
Compare the hidden units across the three starting guesses



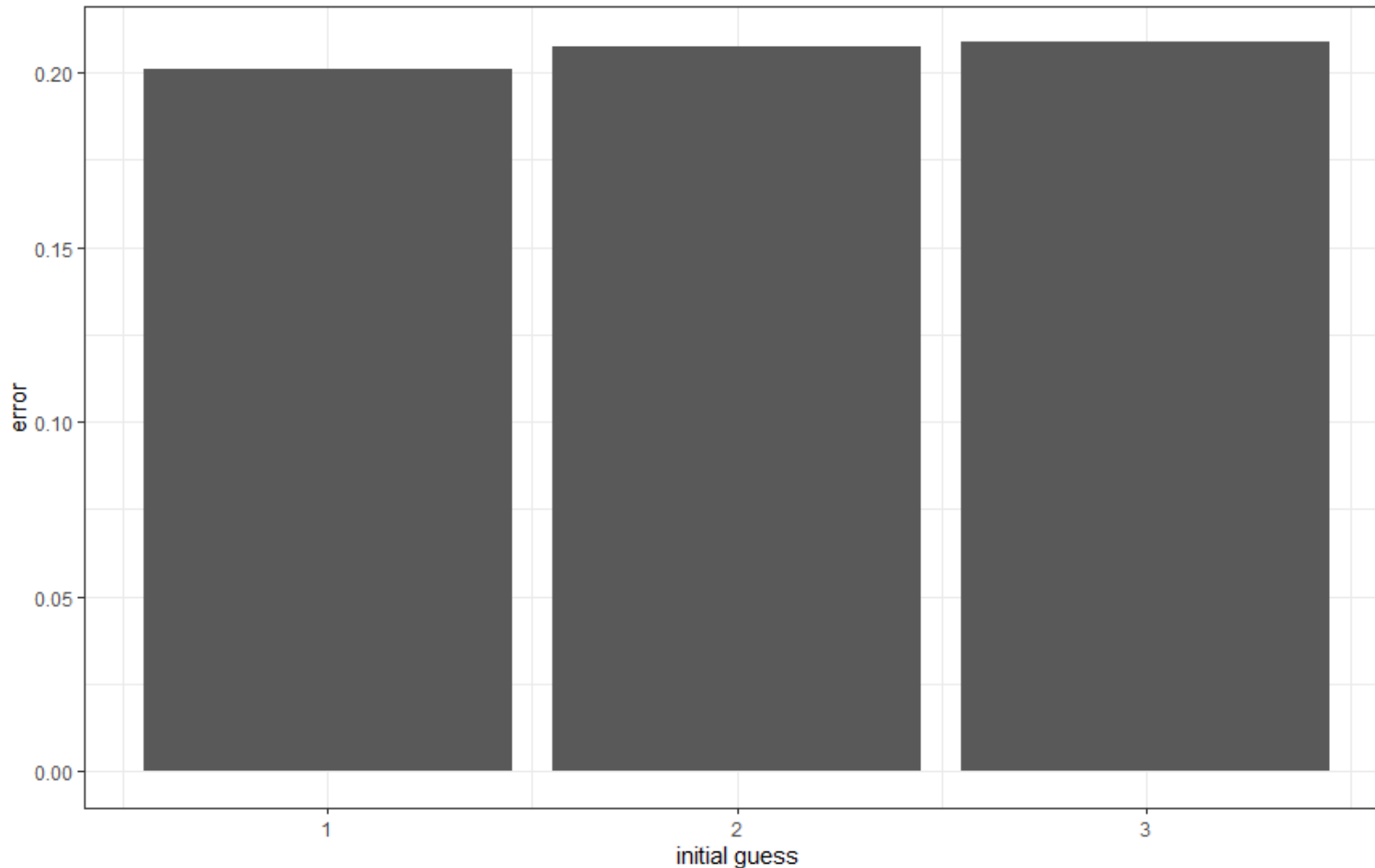
The hidden units are different because the hidden unit parameters are different!



But which model is better?

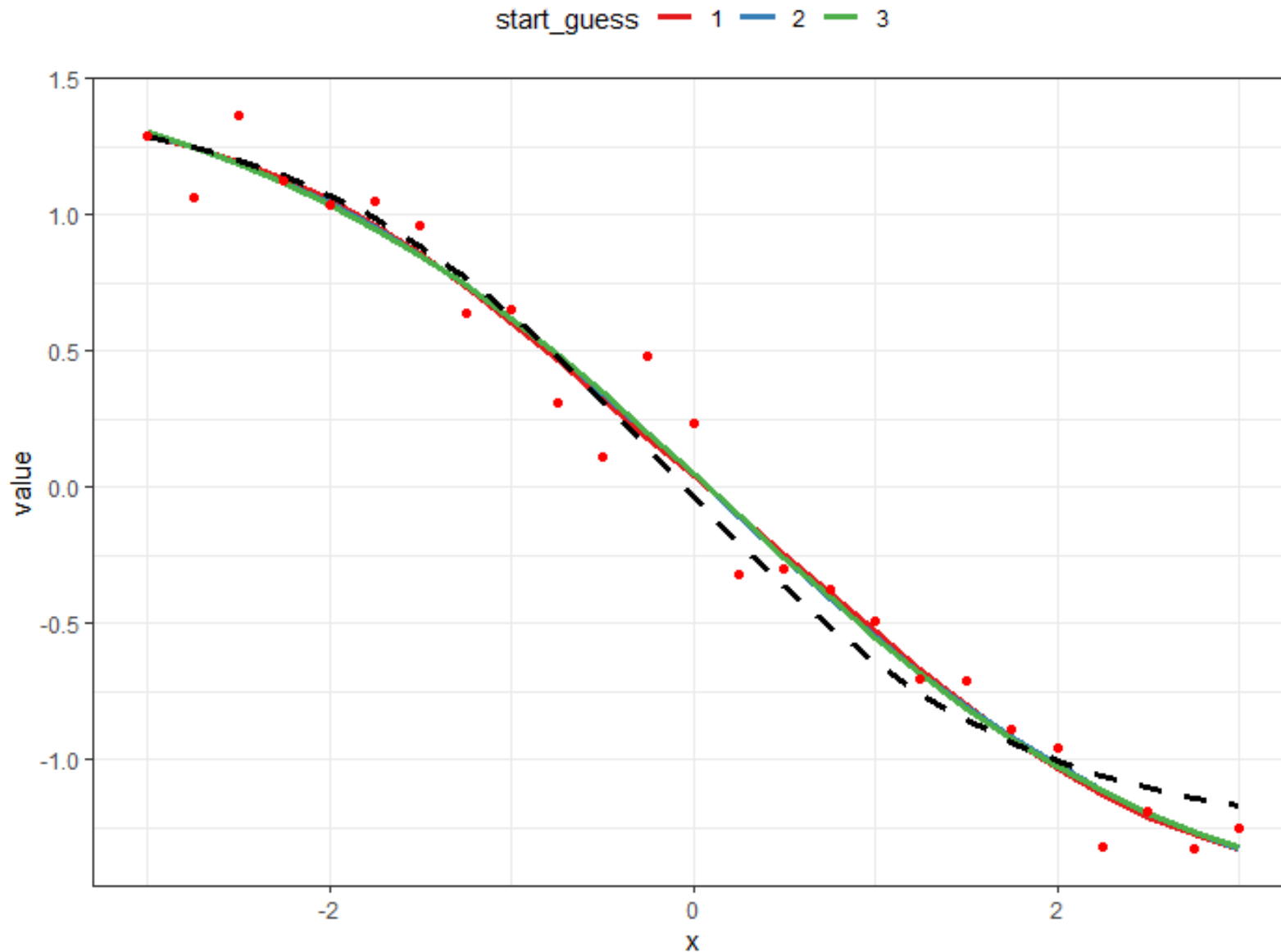


Errors associated with the three different initial guess values

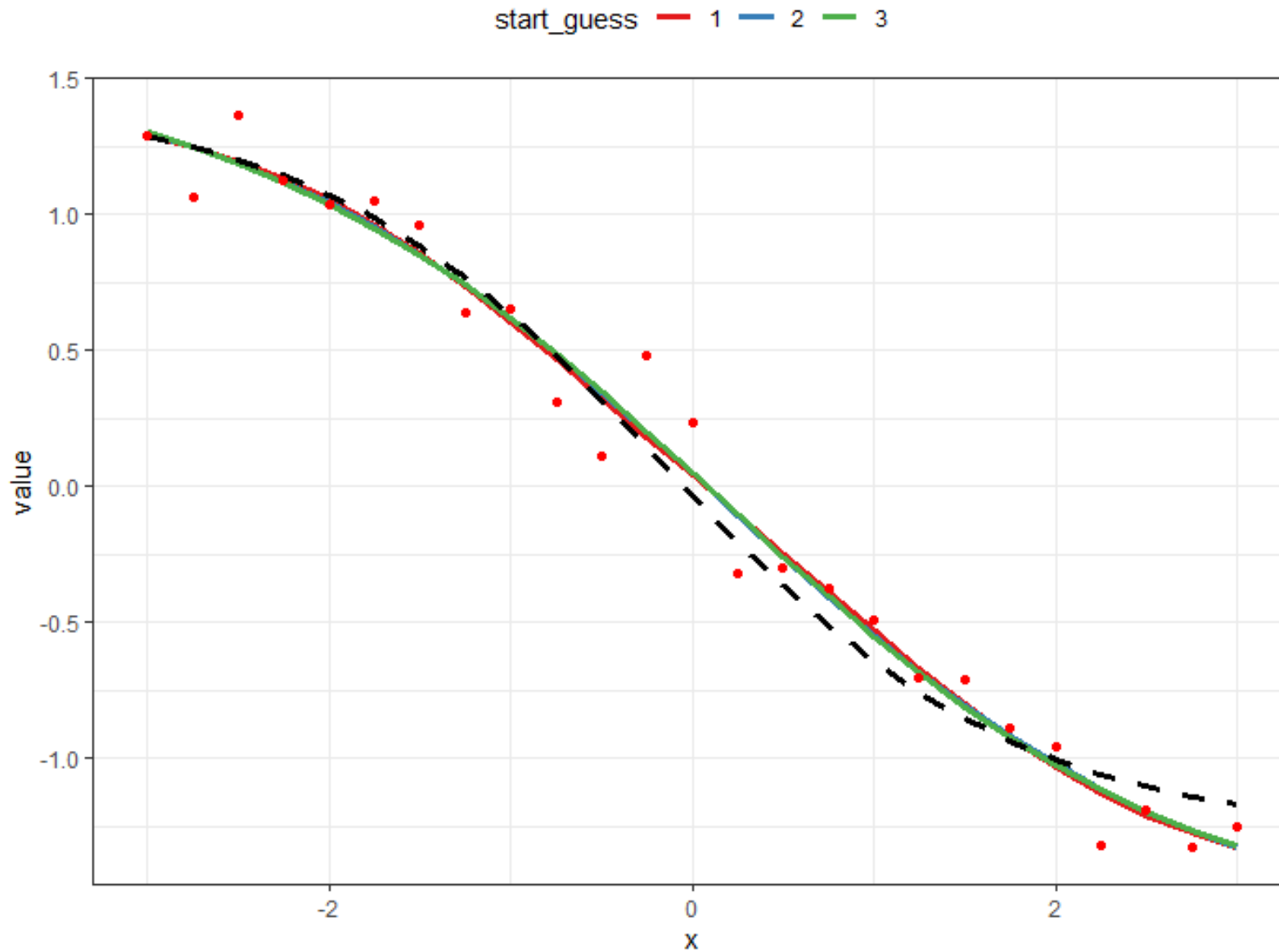


Or, how will the neural network predictions vary across the 3 starting guesses?

The three models are basically the same in this example!



The hidden units are different...why are the results the same?



The response depends on the hidden unit parameters AND the output layer parameters!

```
> mod1_a$weights
[[1]]
[[1]][[1]]
      [,1]      [,2]
[1,]  1.686682 -1.062303
[2,] -1.253870 -1.173721

[[1]][[2]]
      [,1]
[1,] -1.491932
[2,]  1.335982
[3,]  1.570633
```

β

α

```
> mod1_b$weights
[[1]]
[[1]][[1]]
      [,1]      [,2]
[1,] -0.07120246 -0.4755229
[2,] -0.78992667  0.7335158

[[1]][[2]]
      [,1]
[1,] -0.4974662
[2,]  2.0596867
[3,] -1.1749833
```

β

α

```
> mod1_c$weights
[[1]]
[[1]][[1]]
      [,1]      [,2]
[1,]  0.9980648  0.3845942
[2,]  0.7370192 -0.8546132

[[1]][[2]]
      [,1]
[1,] -0.7992029
[2,] -0.7961137
[3,]  2.4058118
```

β

α

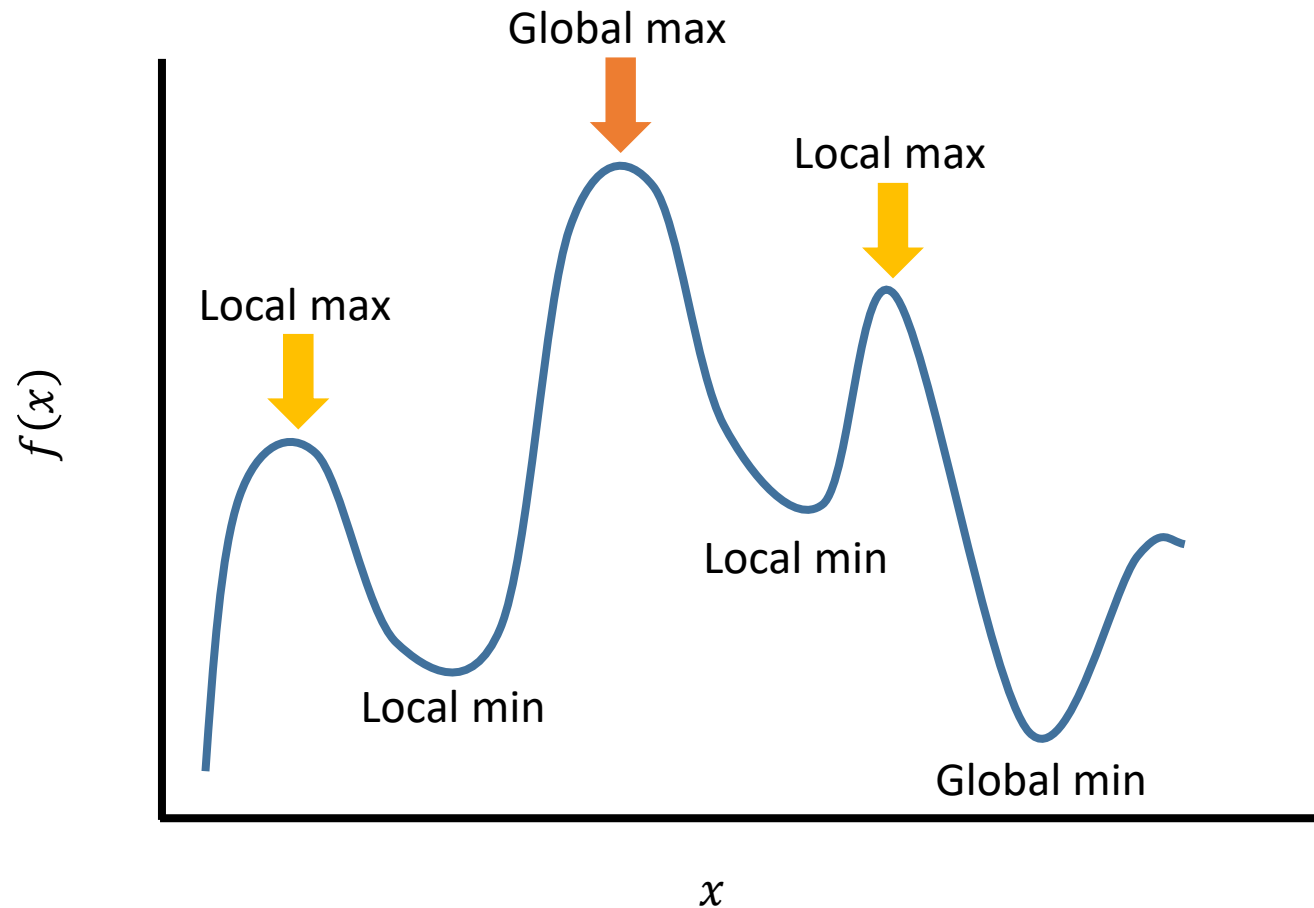
Why does this matter?

- With linear and logistic regression, we saw that our initial guesses do **not** impact the final result.
 - We will ALWAYS identify the same parameter estimates, regardless of our starting guess.
- However, with neural networks the initial guess clearly had an impact on the parameters, and thus impacted the hidden units.

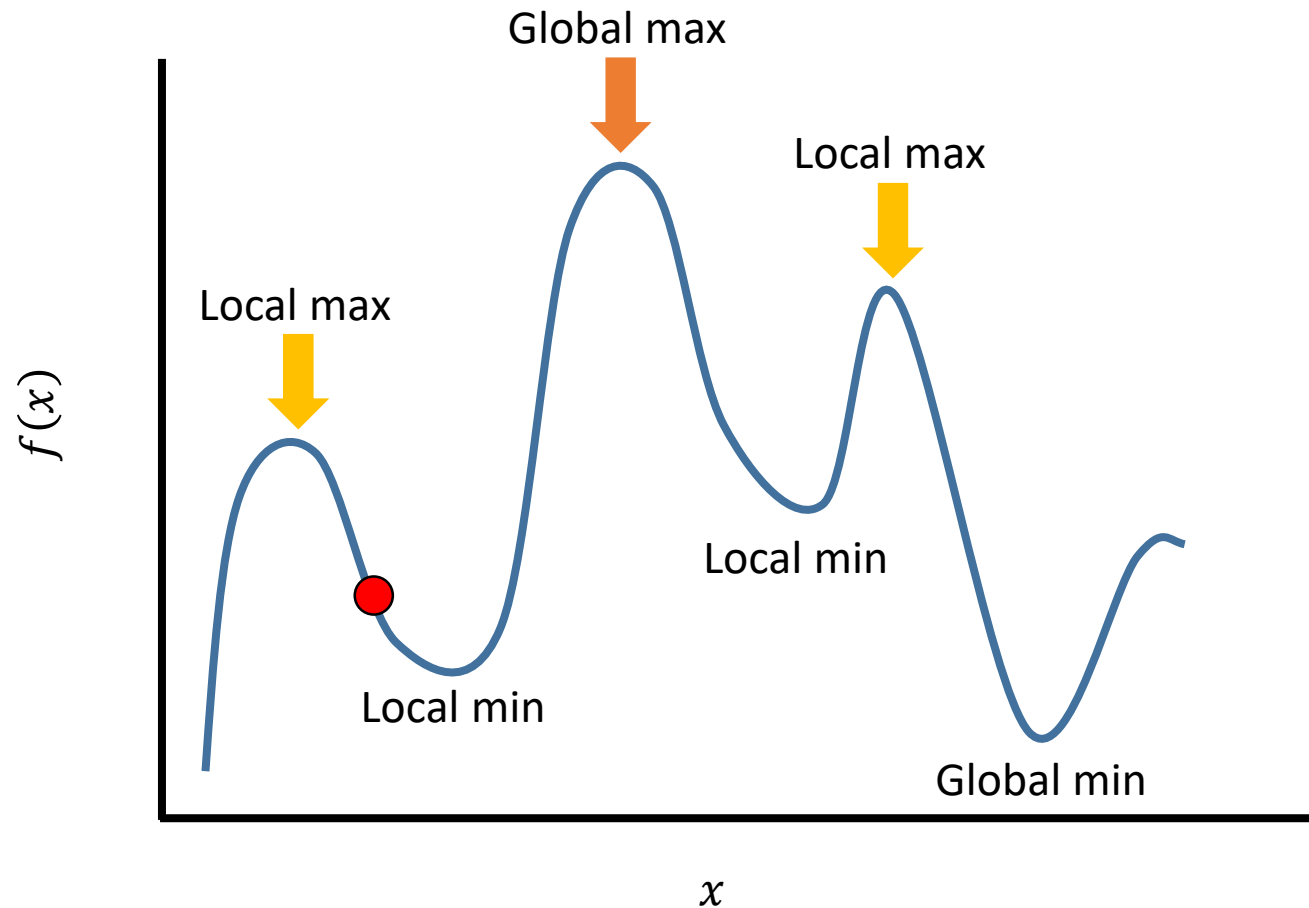
Although it is difficult to visualize, the neural network likelihood surface is MULTIMODAL

- The likelihood consists of many local optima!
- The optimization scheme can get “stuck” at local optima.
- The global optimum and thus “best” parameter estimates can therefore be missed!

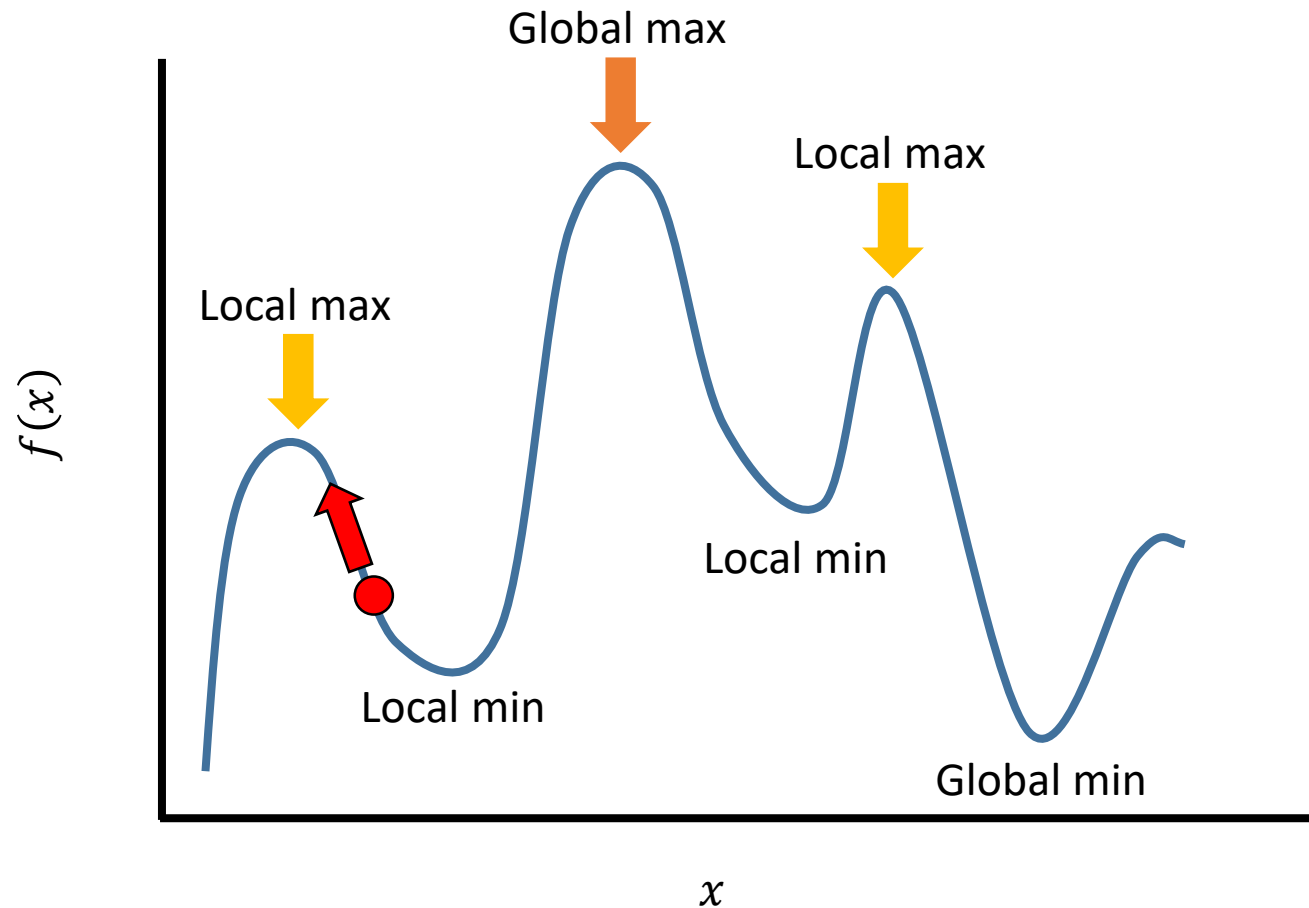
What does it mean to be “stuck”? Consider trying to MAXIMIZE the function depicted below



The initial guess is denoted by the red dot.
Will we find the global max?

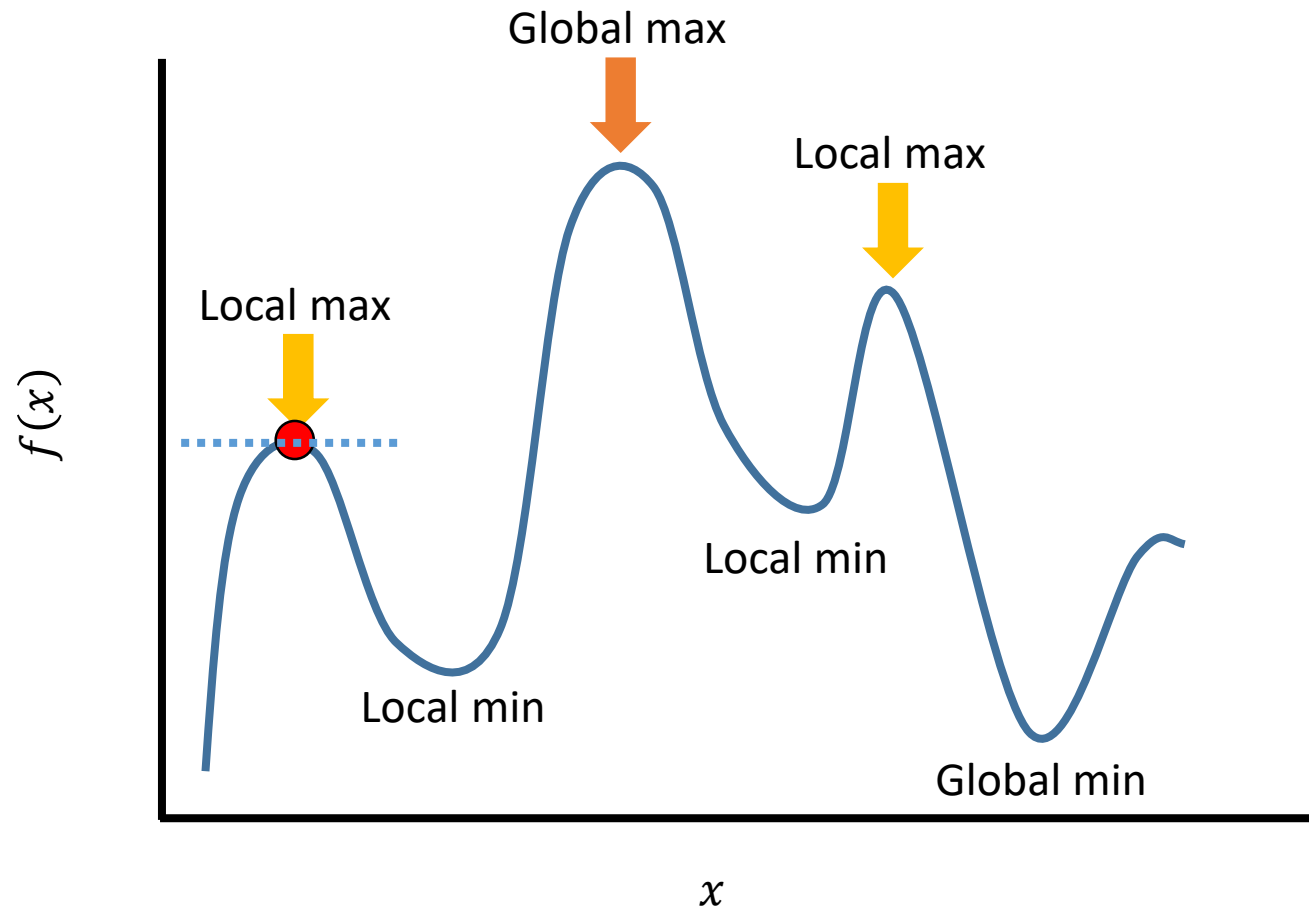


The initial guess is denoted by the red dot.
Will we find the global max?



NO! The gradient points to a LOCAL max.

The optimization algorithm will stop at the local maximum because the gradient equals zero.



Many algorithms have been developed to try and limit or overcome getting stuck at a local optimum

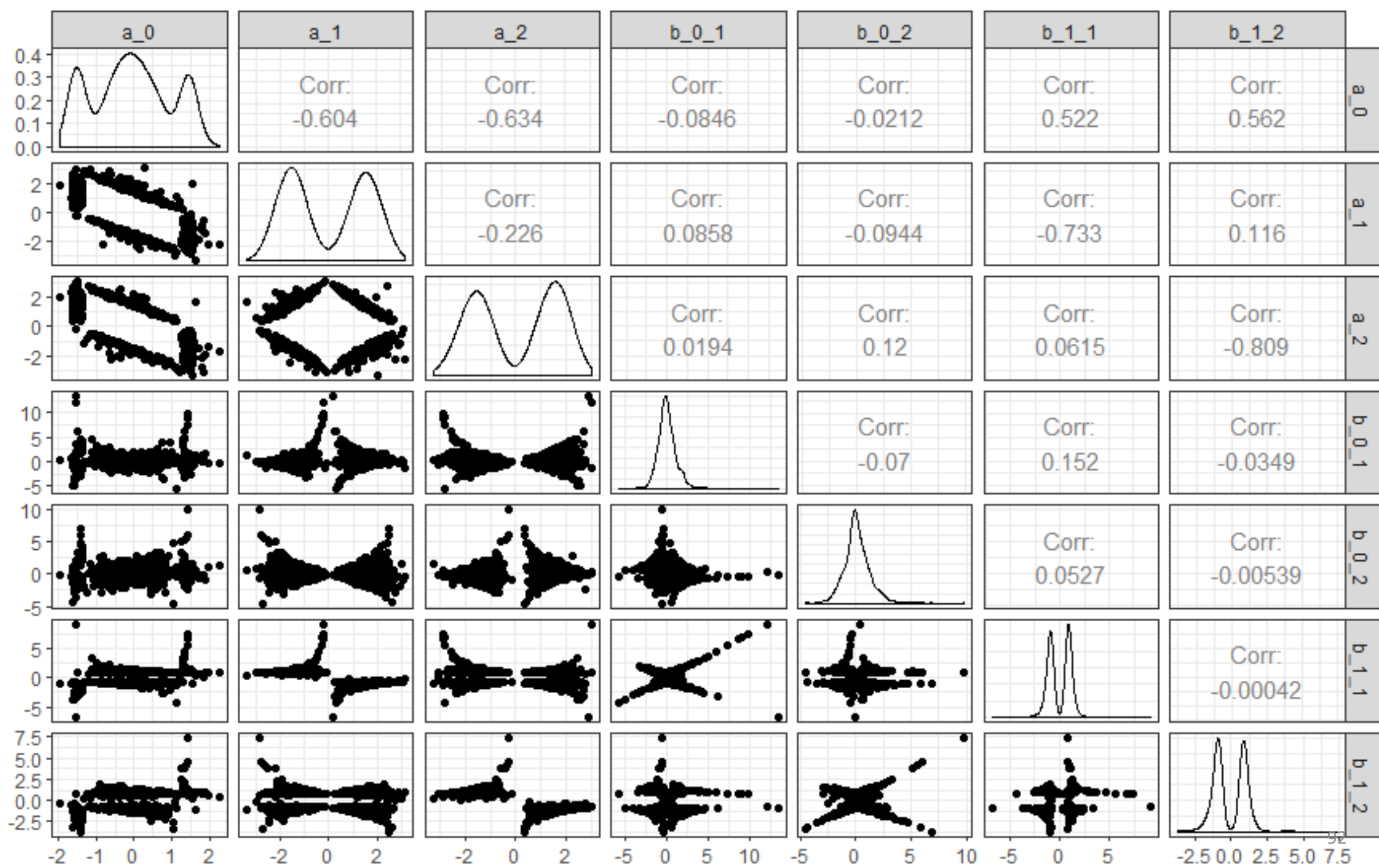
- Even with all of the various advancements, there is no guarantee you will NOT get stuck at a local optimum.
- As our simple toy demo shows though, that is not necessarily a bad thing. Often the local points are good enough.
- Sometimes, the final model results are essentially the same, as was the case in our simple example.

Many algorithms have been developed to try and limit or overcome getting stuck at a local optimum

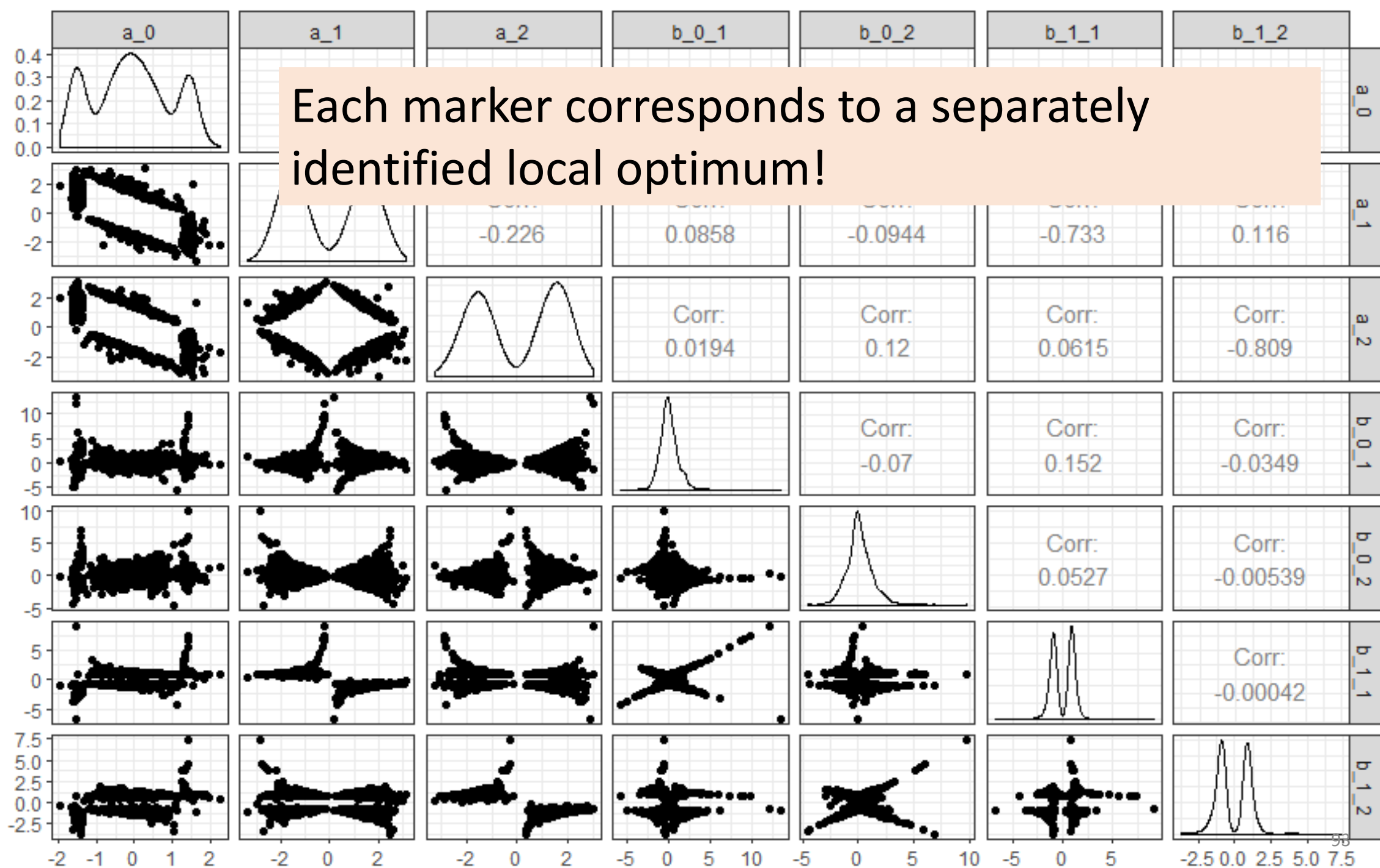
- Even with all of the various advancements, there is no guarantee you will NOT get stuck at a local optimum.
- **HOWEVER**, since we may not know what “good enough” means...lots of effort has gone into developing efficient solutions to find the “best” possible neural network parameters.

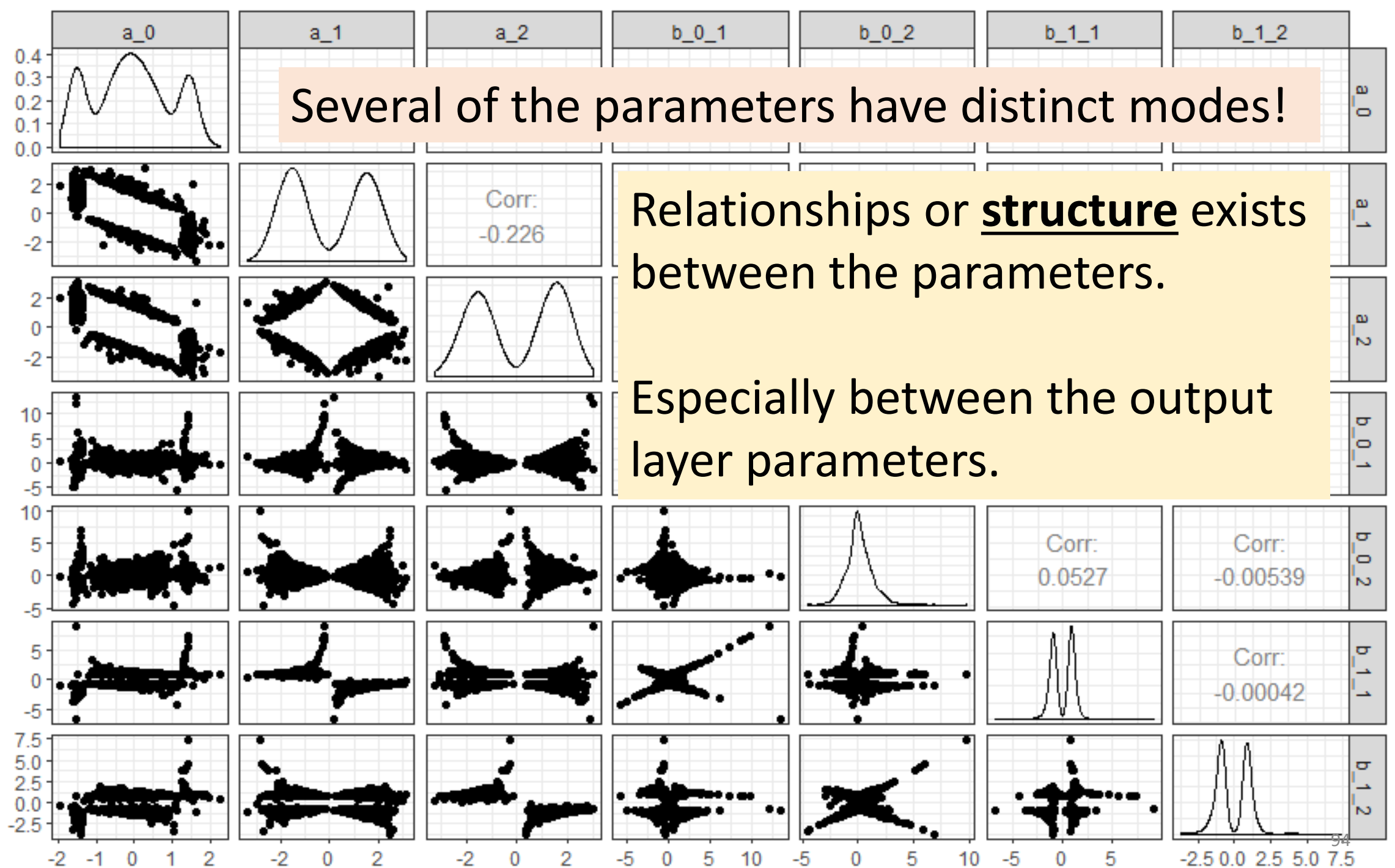
Let's see what happens if we train the neural network for our simple demo many times

- Fit the neural network 1000 times using 1000 different initial guesses.
- Record the fitted values for all 7 parameters.
- **Corresponds to 1000 sets of optimal parameter estimates.**
- Next slide shows the pairs plot matrix for the 7 parameters based on the 1000 different initial guesses.



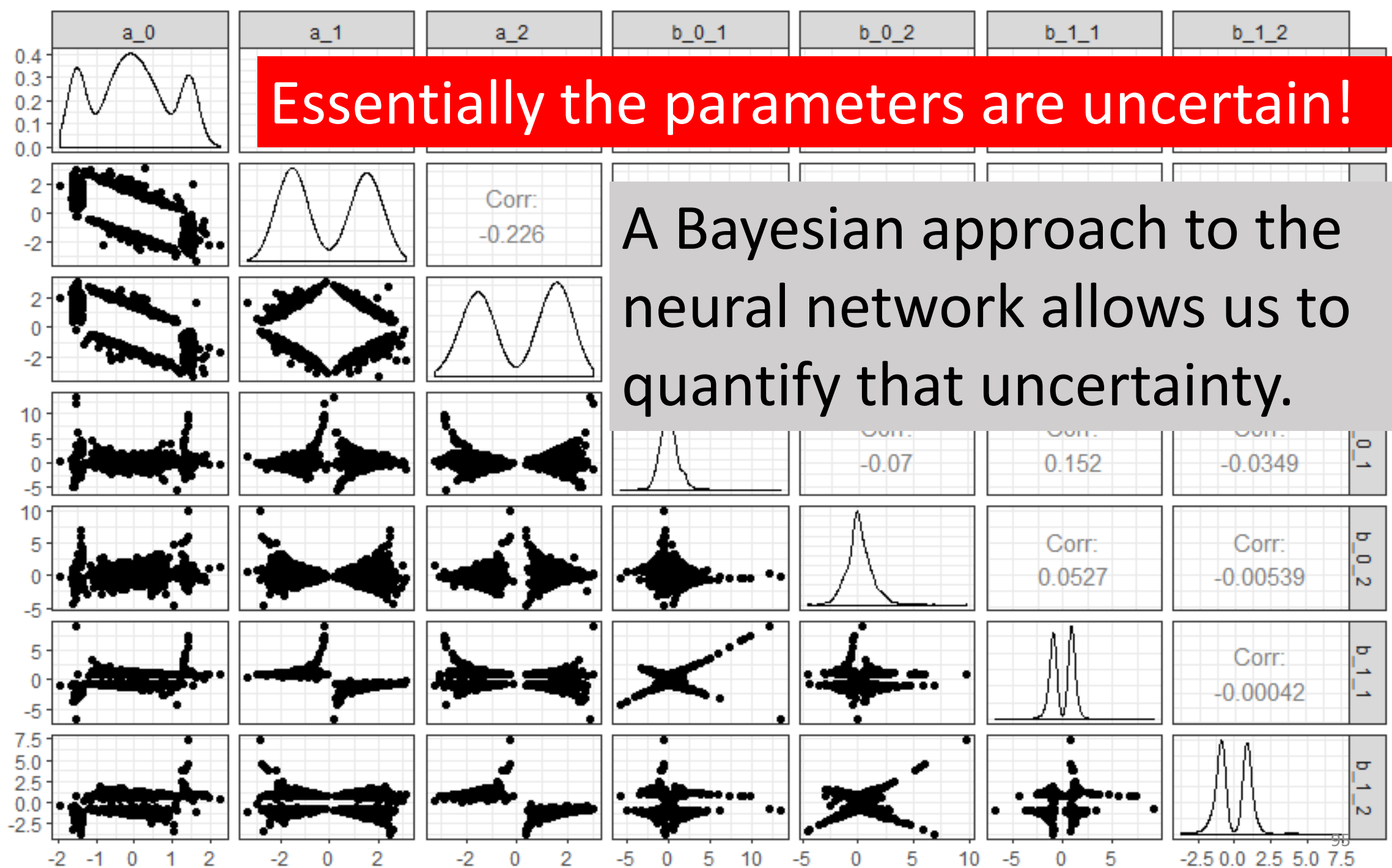
Each marker corresponds to a separately identified local optimum!

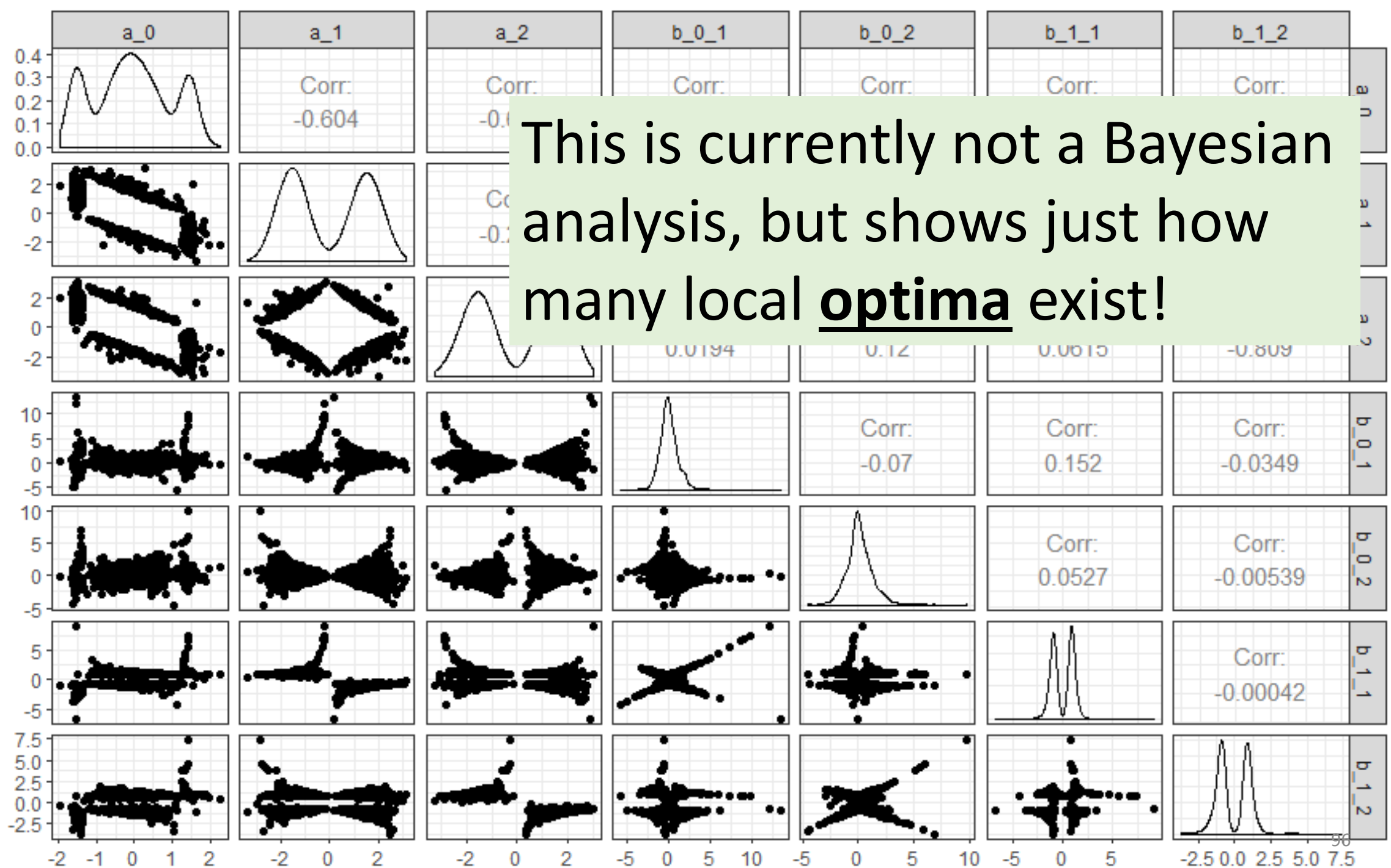




Essentially the parameters are uncertain!

A Bayesian approach to the neural network allows us to quantify that uncertainty.

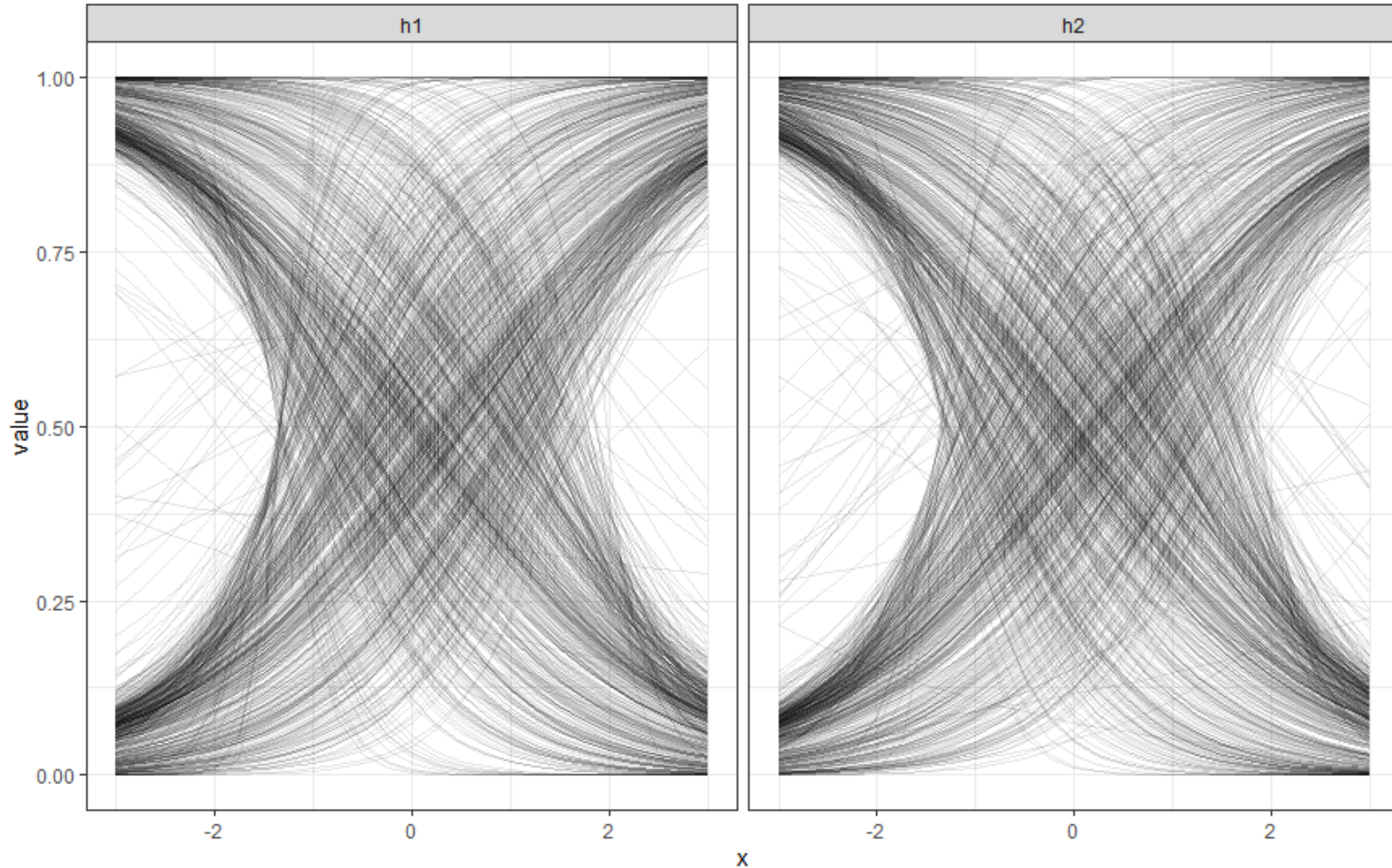




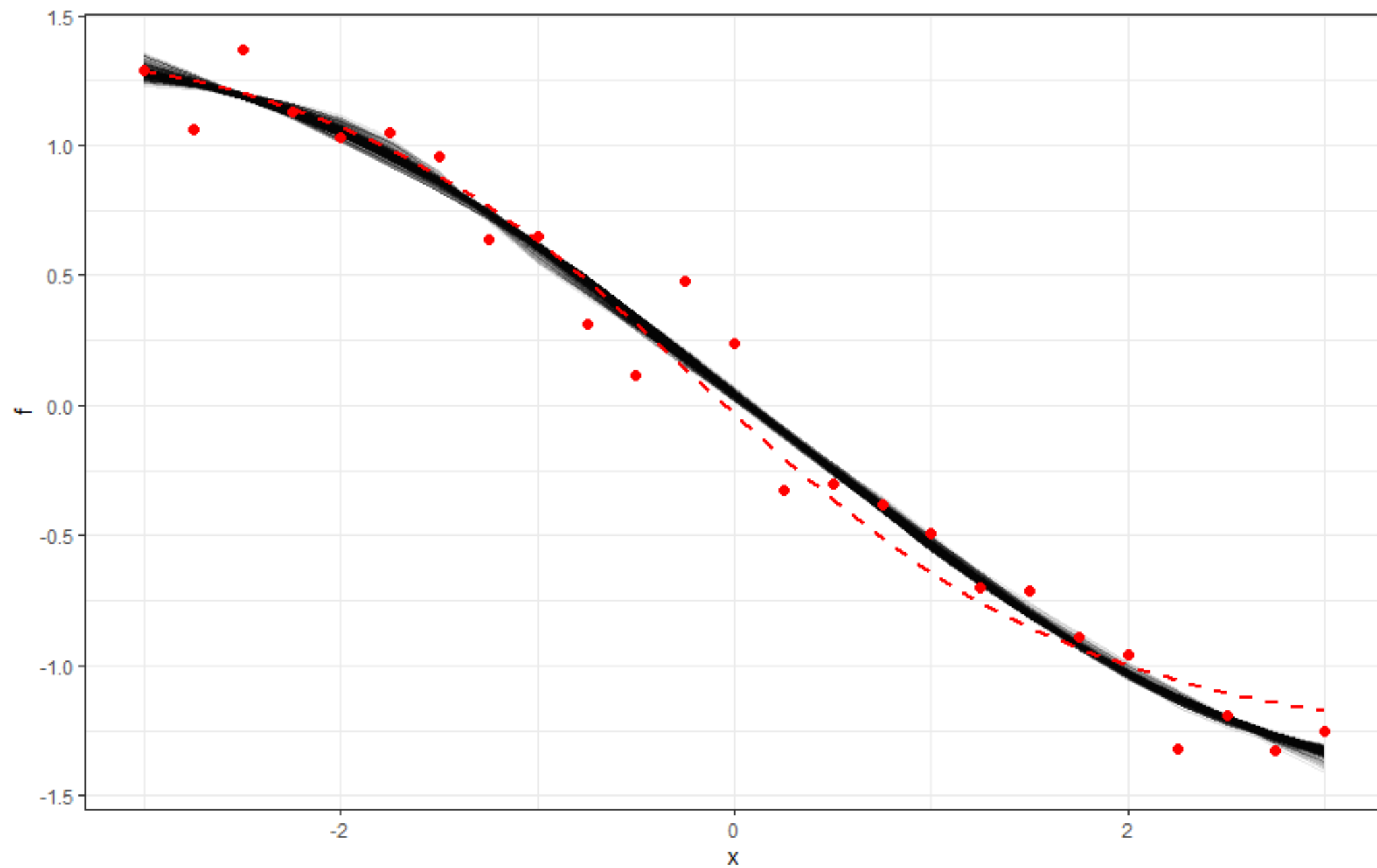
With that many locally optimal parameter estimates, how varied will the hidden units be?

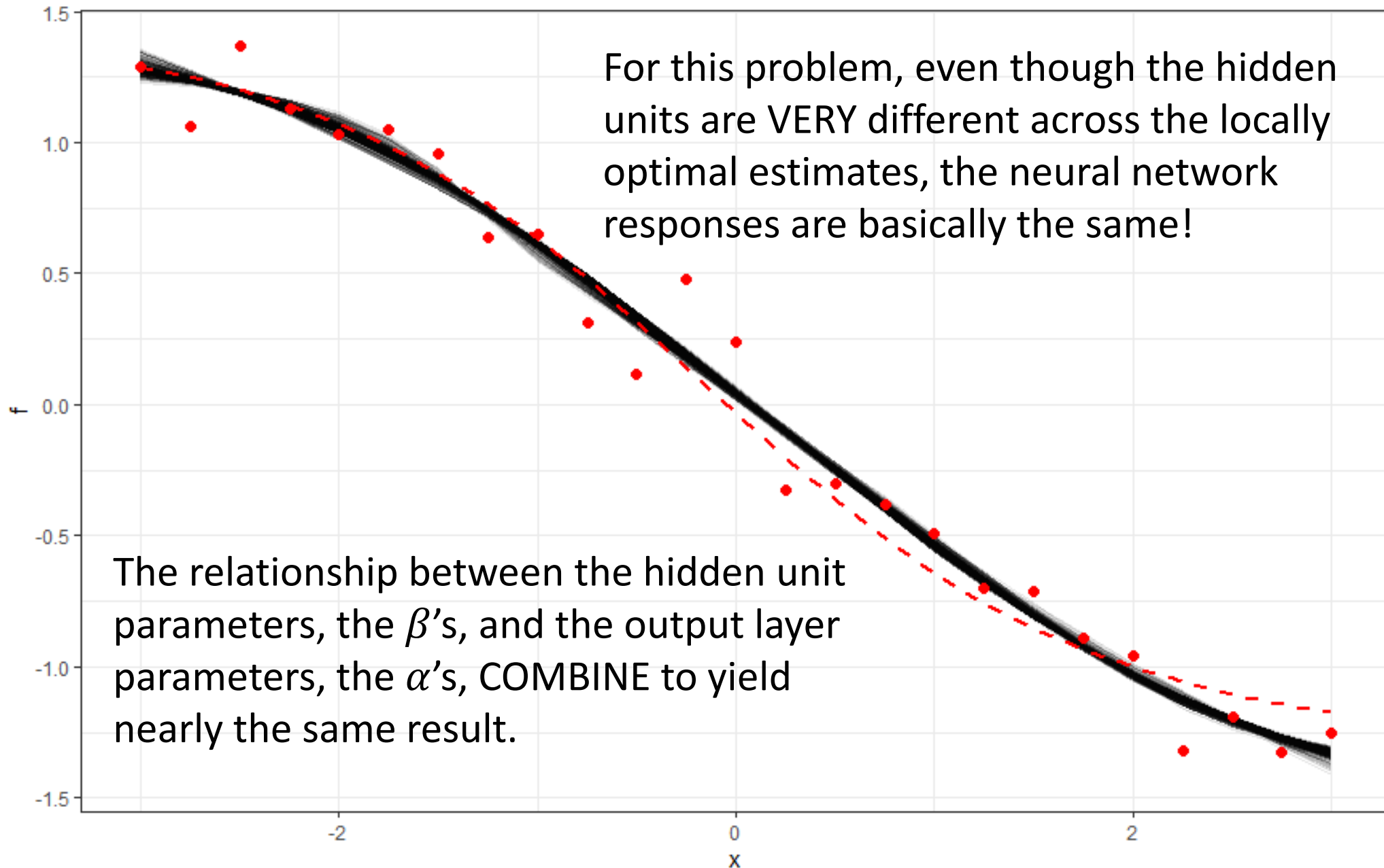
- The next slide shows a plot of the hidden unit responses associated with each of the 1000 identified local optimal parameter sets.

Hidden unit responses with respect to the input



And finally, the variability of the neural network response?





What's the big deal?

- We described the hidden units as the neural network learning a basis or set of features for us.
- The neural network thus adapts the features to the data.
- However, **the feature behavior depends on the initial guess!**

The neural network hidden units, the features, are not easily interpretable

- Just consider how many possible ways we could try and interpret the hidden units in our simple example...
- Is this bad? We are giving up interpretability for greater flexibility and predictive accuracy.
- Depends on your goal!

